

Pulse

A camera pulse reading that refuses to answer when it cannot answer well.

Source code in this document — 2,299 lines

SIGNAL PROCESSING AND THE QUALITY GATE

ppg.py	192 lines
butppg_io.py	52 lines

VALIDATION

validate.py	140 lines
sweep.py	53 lines

PROVING THE BROWSER MATCHES THE PYTHON

verify_dsp.mjs	323 lines
----------------	-----------

THE WEB APPLICATION

web/dsp.js	536 lines
web/app.js	587 lines
web/index.html	51 lines
web/app.css	365 lines

Not included. Corpora under `data/` (PhysioNet, not ours to redistribute), `.venv/` and `node_modules/`, generated caches, and the trained weights `murmur.onnx` / `murmur_net.pt`, which are binary and are produced by `train.py` and `export.py` above.

```

1  """Pulse rhythm from a photoplethysmogram.
2
3  The whole method is arithmetic you can check by hand, on purpose. A model that
4  says "irregular" without showing why is useless to a clinician, and a student
5  project that cannot explain its own maths is worse.
6
7  Pipeline: detrend, bandpass to plausible heart rates, find beats, measure the
8  intervals between them, then judge the spread of those intervals.
9  """
10
11 from __future__ import annotations
12
13 import numpy as np
14 from scipy.signal import butter, filtfilt, find_peaks
15
16 #: Adult heart rates live between about 30 and 200 bpm. Anything outside that
17 #: is noise, not a pulse, and letting it through is how a shaky hand becomes a
18 #: diagnosis.
19 HR_MIN_BPM, HR_MAX_BPM = 30.0, 200.0
20 #: Chosen on BUT PPG against heart rates derived from the simultaneous ECG's
21 #: R-peak annotations -- not against the dataset's own HR column, which is
22 #: doubled for one subject and differs from the R-peaks by 2.98 bpm on average.
23 #: Raising it refuses more, which is the safe direction: a wrong rate is worse
24 #: than no rate. At 0.85, with the cleanest-window selection below, the tool
25 #: accepts 5.3% of the corpus and 85.1% of those rates land within 5 bpm of the
26 #: ECG. (Measuring a fixed ten-second window instead gives 3.5% and 90%: fewer
27 #: recordings, each a little more accurate. See validation.json.)
28 PURITY_MIN = 0.85
29 #: Peak-counting and the spectrum are independent estimates of the same rate.
30 #: Requiring them to agree closely is the single most effective check here:
31 #: it is what catches the half-rate errors that come from missed alternate
32 #: beats, which are otherwise indistinguishable from a genuine bradycardia.
33 AGREEMENT_TOL = 0.05
34 LOW_HZ, HIGH_HZ = HR_MIN_BPM / 60.0, HR_MAX_BPM / 60.0
35
36
37 def bandpass(x: np.ndarray, fs: float) -> np.ndarray:
38     ny = fs / 2.0
39     lo, hi = LOW_HZ / ny, min(HIGH_HZ / ny, 0.99)
40     b, a = butter(3, [lo, hi], btype="band")
41     return filtfilt(b, a, x).astype(np.float64)
42
43
44 def detrend_normalise(x: np.ndarray) -> np.ndarray:
45     x = np.asarray(x, dtype=np.float64)
46     x = x - np.mean(x)
47     s = np.std(x)
48     return x / s if s > 0 else x
49
50
51 def find_beats(x: np.ndarray, fs: float) -> np.ndarray:
52     """Beat sample indices.
53
54     Minimum spacing comes from the maximum plausible rate, so one beat cannot
55     be counted twice. Prominence is relative to the signal's own spread, so it
56     adapts to a weak trace instead of needing a magic constant.
57     """
58     min_gap = int(fs * 60.0 / HR_MAX_BPM)
59     prom = 0.3 * np.std(x)
60     peaks, _ = find_peaks(x, distance=max(min_gap, 1), prominence=prom)
61     return peaks
62
63
64 def intervals_ms(peaks: np.ndarray, fs: float) -> np.ndarray:
65     if peaks.size < 2:
66         return np.array([])
67     return np.diff(peaks) / fs * 1000.0
68
69
70 def irregularity(ibi: np.ndarray) -> dict:
71     """Standard beat-to-beat variability measures.
72

```

```

73     rmssd and pnn50 are the usual descriptors of how much consecutive beats
74     differ. Atrial fibrillation is irregularly irregular, so both run high;
75     a steady rhythm keeps them low. These are descriptions of the trace, not
76     a diagnosis of the heart.
77     """
78     if ibi.size < 3:
79         return {"n_beats": int(ibi.size + 1) if ibi.size else 0}
80     diffs = np.diff(ibi)
81     rmssd = float(np.sqrt(np.mean(diffs ** 2)))
82     pnn50 = float(np.mean(np.abs(diffs) > 50.0))
83     mean_ibi = float(np.mean(ibi))
84     return {
85         "n_beats": int(ibi.size + 1),
86         "mean_ibi_ms": mean_ibi,
87         "hr_bpm": 60000.0 / mean_ibi if mean_ibi > 0 else 0.0,
88         "sdnn_ms": float(np.std(ibi)),
89         "rmssd_ms": rmssd,
90         "pnn50": pnn50,
91         "cv": float(np.std(ibi) / mean_ibi) if mean_ibi > 0 else 0.0,
92     }
93
94
95 def spectral_purity(x: np.ndarray, fs: float) -> tuple[float, float]:
96     """How much of the in-band power sits in one peak, and where that peak is.
97
98     A real pulse is close to periodic, so its spectrum has one dominant lobe in
99     the heart-rate band. A trace dominated by movement smears across the band.
100    This turns out to separate measurable from unmeasurable far better than
101    anything measured in the time domain.
102    """
103    n = x.size
104    win = np.hanning(n)
105    spec = np.abs(np.fft.rfft(x * win)) ** 2
106    freqs = np.fft.rfftfreq(n, 1.0 / fs)
107    band = (freqs >= LOW_HZ) & (freqs <= HIGH_HZ)
108    if not band.any() or spec[band].sum() <= 0:
109        return 0.0, 0.0
110    bp = spec[band]
111    bf = freqs[band]
112    k = int(np.argmax(bp))
113    # the peak plus its immediate neighbours, against all in-band power
114    lo, hi = max(k - 1, 0), min(k + 2, bp.size)
115    return float(bp[lo:hi].sum() / bp.sum()), float(bf[k] * 60.0)
116
117
118 def signal_quality(x: np.ndarray, peaks: np.ndarray, fs: float) -> dict:
119     """Can this trace be measured at all?
120
121     Most smartphone PPG recordings cannot. In BUT PPG, 3,058 of 3,888 records
122     are annotated poor quality. Refusing is the common correct answer, so the
123     tool has to be good at it rather than treat it as an edge case.
124     """
125     # Four peaks, not three: three peaks give two intervals, and the spread of
126     # two numbers says nothing about rhythm. A trace that cannot support the
127     # measurement should not pass the gate that guards it.
128     if peaks.size < 4:
129         return {"usable": False, "reason": "too few beats detected", "beats": int(peaks.size)}
130     ibi = intervals_ms(peaks, fs)
131     hr = 60000.0 / np.mean(ibi)
132     if not (HR_MIN_BPM <= hr <= HR_MAX_BPM):
133         return {"usable": False, "reason": f"implied rate {hr:.0f} bpm is outside 30-200", "beats": int(peaks.size)}
134     # A trace whose beats vary wildly in height is usually a moving finger
135     purity, peak_bpm = spectral_purity(x, fs)
136     if purity < PURITY_MIN:
137         return {"usable": False, "beats": int(peaks.size), "purity": purity,
138             "reason": "no single dominant pulse frequency; likely movement or poor contact"}
139     # the beats we counted should agree with the frequency the spectrum found
140     if peak_bpm > 0 and abs(hr - peak_bpm) / peak_bpm > AGREEMENT_TOL:
141         return {"usable": False, "beats": int(peaks.size), "purity": purity,
142             "reason": f"counted {hr:.0f} bpm but the spectrum says {peak_bpm:.0f} bpm"}
143     return {"usable": True, "reason": "", "beats": int(peaks.size), "purity": purity}
144
145
146 #: filtfilt needs a few times the filter order in samples. A trace shorter than
147 #: this cannot be filtered, let alone measured, so it is refused rather than

```

```

148 #: forced through -- the same answer the tool gives a human holding still badly.
149 MIN_SECONDS = 4.0
150 #: Accuracy is duration-limited rather than method-limited: on the same gate,
151 #: the share of rates within 5 bpm of the ECG runs 61% at a 4 s window, 74% at
152 #: 6 s, 82% at 8 s and 90% at 10 s. Longer is better, but a longer clip also
153 #: has more opportunity to contain movement, so the tool records generously and
154 #: measures the calmest stretch it can find rather than whatever it began with.
155 WINDOW_SECONDS = 8.0
156 WINDOW_STEP_SECONDS = 0.5
157
158
159 def analyse(raw: np.ndarray, fs: float) -> dict:
160     raw = np.asarray(raw, dtype=np.float64).ravel()
161     if raw.size < max(int(MIN_SECONDS * fs), 25):
162         return {"fs": fs, "peaks": [], "quality": {
163             "usable": False,
164             "reason": f"recording is {raw.size / fs:.1f} s; needs at least {MIN_SECONDS:.0f} s",
165             "beats": 0}}
166     n = int(WINDOW_SECONDS * fs)
167     if raw.size <= n:
168         offsets = [0]
169         n = raw.size
170     else:
171         step = max(int(WINDOW_STEP_SECONDS * fs), 1)
172         offsets = list(range(0, raw.size - n + 1, step))
173
174     # Every candidate window is scored; the calmest one is the one we report.
175     # Picking by purity rather than by the rate it produces keeps the choice
176     # independent of the answer, so this cannot quietly select for a number.
177     best = None
178     for off in offsets:
179         x = bandpass(detrend_normalise(raw[off:off + n]), fs)
180         peaks = find_beats(x, fs)
181         q = signal_quality(x, peaks, fs)
182         score = q.get("purity", -1.0)
183         if best is None or score > best[0]:
184             best = (score, off, x, peaks, q)
185
186     _, off, x, peaks, q = best
187     out = {"fs": fs, "quality": q, "peaks": peaks.tolist(),
188           "window": {"start_s": off / fs, "seconds": n / fs,
189                     "of_seconds": raw.size / fs}}
190     if q["usable"]:
191         out.update(irregularity(intervals_ms(peaks, fs)))
192     return out

```

```

1  """Read BUT PPG records: WFDB headers, 16-bit signals, and QRS reference beats."""
2  from __future__ import annotations
3  import numpy as np
4  from pathlib import Path
5
6  def read_header(hea: Path) -> dict:
7      lines = [l.strip() for l in hea.read_text().splitlines() if l.strip() and not l.startswith("#")]
8      name, nsig, fs, nsamp = lines[0].split()[:4]
9      sigs = []
10     for l in lines[1:1 + int(nsig)]:
11         f = l.split()
12         gain = f[2].split("(")[0].split("/")[0]
13         sigs.append({"file": f[0], "gain": float(gain) if gain else 1.0,
14                     "baseline": int(f[2].split("(")[1].split(")")[0]) if "(" in f[2] else 0,
15                     "name": f[-1]})
16     return {"name": name, "nsig": int(nsig), "fs": float(fs), "nsamp": int(nsamp), "sigs": sigs}
17
18 def read_signal(hea: Path) -> tuple[np.ndarray, dict]:
19     h = read_header(hea)
20     # baselines run to hundreds of thousands; subtracting them in int16 overflows
21     raw = np.fromfile(hea.with_suffix(".dat"), dtype="<i2").astype(np.float64)
22     n = h["nsamp"]; k = h["nsig"]
23     raw = raw[: n * k].reshape(n, k)
24     out = np.empty((n, k), dtype=np.float64)
25     for i, s in enumerate(h["sigs"]):
26         out[:, i] = (raw[:, i] - s["baseline"]) / (s["gain"] or 1.0)
27     return out, h
28
29 def read_qrs(path: Path) -> np.ndarray:
30     """WFDB annotation file -> sample indices of reference beats."""
31     b = np.fromfile(path, dtype=np.uint8).astype(np.int64) # avoid uint8 overflow in the bit maths
32     pos, i, t = [], 0, 0
33     while i + 1 < len(b):
34         word = b[i] | (b[i + 1] << 8)
35         code, interval = word >> 10, word & 0x3FF
36         i += 2
37         if code == 59 and interval == 0: # SKIP: 4-byte absolute interval
38             if i + 3 >= len(b): break
39             interval = (b[i + 1] << 24) | (b[i] << 16) | (b[i + 3] << 8) | b[i + 2]
40             i += 4
41             code = (b[i] | (b[i + 1] << 8)) >> 10 if i + 1 < len(b) else 0
42             i += 2
43         elif code in (60, 61, 62, 63): # NUM/SUB/CHN/AUX payloads
44             if code == 63:
45                 i += interval + (interval % 2)
46             continue
47         if code == 0 and interval == 0:
48             break
49         t += interval
50         if 1 <= code <= 13: # beat annotations
51             pos.append(t)
52     return np.array(pos, dtype=int)

```

```

1  """Measure the pipeline against BUT PPG's reference annotations.
2
3  Two questions, both honest:
4      1. When the trace is measurable, is the rate right? Compared against a heart
5         rate computed from the R-peak annotations on the simultaneous ECG, over
6         the same window. Not against the dataset's own HR column: that column is
7         doubled for subject 142 and sits 2.98 bpm from the R-peaks on average,
8         so scoring against it charges us for the reference's errors as well.
9      2. Does the tool know when it cannot measure? 3,058 of 3,888 records here are
10         annotated poor quality, so refusing well matters more than measuring well.
11 """
12 from __future__ import annotations
13 import csv, json
14 from pathlib import Path
15 import numpy as np
16 import butppg_io as io, ppg
17
18 HERE = Path(__file__).resolve().parent
19 ROOT = next(HERE.glob("data/butppg/**/*_PPG.he*")).parent.parent
20
21 def ecg_rate(rec: Path, key: str) -> float | None:
22     """Heart rate from the ECG's own R-peak annotations, or None."""
23     qrs = rec / f"{key}.qrs"
24     if not qrs.exists():
25         return None
26     try:
27         h = io.read_header(rec / f"{key}_ECG.he*")
28         peaks = io.read_qrs(qrs)
29     except Exception:
30         return None
31     if peaks is None or len(peaks) < 4:
32         return None
33     ibi = np.diff(np.asarray(peaks, dtype=np.float64)) / h["fs"] * 1000.0
34     # 250-2500 ms spans 24-240 bpm; anything outside is a detector artefact
35     ibi = ibi[(ibi > 250.0) & (ibi < 2500.0)]
36     return float(60000.0 / np.mean(ibi)) if ibi.size >= 3 else None
37
38
39 def annotations() -> dict:
40     out = {}
41     with (HERE / "data" / "quality-hr-ann.csv").open(encoding="utf-8-sig") as f:
42         for r in csv.DictReader(f):
43             try: out[r["ID"].strip()] = (int(r["Quality"]), float(r["HR"]))
44             except (ValueError, KeyError): pass
45     return out
46
47 def main():
48     ann = annotations()
49     recs = sorted(p for p in ROOT.iterdir() if p.is_dir())
50     rows = []
51     for rec in recs:
52         key = rec.name
53         if key not in ann: continue
54         hea = rec / f"{key}_PPG.he*"
55         try:
56             h0 = io.read_header(hea)
57             # 48 legacy records encode 300 samples as 300 one-sample "signals".
58             # They are not the camera modality this tool targets, so skip them
59             # rather than silently mangle them into a result.
60             if h0["nsig"] != 3 or h0["nsamp"] < 150:
61                 continue
62             sig, h = io.read_signal(hea)
63         except Exception:
64             continue
65         # green channel tracks blood volume best; fall back when only one exists
66         names = [s["name"] for s in h["sigs"]]
67         ch = names.index("PPG_G") if "PPG_G" in names else 0
68         r = ppg.analyse(sig[:, ch] if sig.ndim > 1 else sig.ravel(), h["fs"])
69         good, _ = ann[key]
70         ref_hr = ecg_rate(rec, key)
71         if ref_hr is None:
72             continue

```

```

73     rows.append({"id": key, "ref_quality": good, "ref_hr": ref_hr,
74                 "usable": bool(r["quality"]["usable"]),
75                 "hr": float(r.get("hr_bpm", 0.0)),
76                 "mssd": float(r.get("mssd_ms", 0.0)),
77                 "cv": float(r.get("cv", 0.0)),
78                 "purity": float(r["quality"].get("purity", 0.0))})
79 (HERE / "rows_cache.json").write_text(json.dumps(rows))
80 n = len(rows)
81 ours = np.array([r["usable"] for r in rows])
82 theirs = np.array([r["ref_quality"] == 1 for r in rows])
83 tp = int((ours & theirs).sum()); tn = int((~ours & ~theirs).sum())
84 fp = int((ours & ~theirs).sum()); fn = int((~ours & theirs).sum())
85
86 print(f"records analysed: {n}")
87 print(f"\nQUALITY GATE vs the annotators")
88 print(f"  they call good : {int(theirs.sum())}   we accept: {int(ours.sum())}")
89 print(f"  agree accept   : {tp}")
90 print(f"  agree refuse   : {tn}")
91 print(f"  we accept, they refuse : {fp}   <- the dangerous direction")
92 print(f"  we refuse, they accept : {fn}   <- merely wasteful")
93 print(f"  specificity (refusing what they refuse): {tn/max(tn+fp,1):.3f}")
94 print(f"  sensitivity (accepting what they accept): {tp/max(tp+fn,1):.3f}")
95
96 kept = [r for r in rows if r["usable"]]
97 err = np.array([abs(r["hr"] - r["ref_hr"]) for r in kept])
98 print(f"\nHEART RATE, on all {len(kept)} records the tool accepts")
99 if len(kept):
100     print(f"  mean absolute error : {err.mean():.2f} bpm")
101     print(f"  median                  : {np.median(err):.2f} bpm")
102     print(f"  within 5 bpm              : {(err<=5).mean()*100:.1f}%")
103     print(f"  within 10 bpm             : {(err<=10).mean()*100:.1f}%")
104 record = {
105     "dataset": "BUT PPG 2.0.0 (PhysioNet, CC-BY)",
106     "reference": "the ECG's own R-peak annotations over the same window",
107     "records_analysed": n,
108     "records_excluded": "48 legacy records encoding 300 samples as 300 one-sample signals",
109     "gate": {"purity_min": ppg.PURITY_MIN, "agreement_tol": ppg.AGREEMENT_TOL,
110             "min_beats": 4, "window_seconds": ppg.WINDOW_SECONDS,
111             "window_step_seconds": ppg.WINDOW_STEP_SECONDS},
112     "quality_gate": {"annotator_good": int(theirs.sum()), "we_accept": int(ours.sum()),
113                     "tp": tp, "tn": tn, "fp": fp, "fn": fn,
114                     "specificity": round(tn/max(tn+fp,1), 3),
115                     "sensitivity": round(tp/max(tp+fn,1), 3)},
116     "heart_rate_on_accepted": {
117         "n": len(kept), "share_of_corpus": round(len(kept)/max(n,1), 4),
118         "mae_bpm": round(float(err.mean()), 2) if len(kept) else None,
119         "median_bpm": round(float(np.median(err)), 2) if len(kept) else None,
120         "within_5_bpm": round(float((err<=5).mean()), 3) if len(kept) else None,
121         "within_10_bpm": round(float((err<=10).mean()), 3) if len(kept) else None},
122     # Measured by sweeping the window length with the gate held fixed. Kept
123     # here rather than recomputed on every run because it takes a full pass
124     # over the corpus per window length.
125     "duration_trend_within_5bpm": {"4s": 0.61, "6s": 0.74, "8s": 0.82, "10s": 0.90},
126     "window_selection": {"fixed_8s": {"n": 149, "mae": 7.33, "within_5": 0.82},
127                          "cleanest_8s": {"n": 204, "mae": 6.38, "within_5": 0.84}},
128     "honest_limits": [
129         "Validated only on 10-second recordings; the app captures 30 s and the duration trend is an extrapolation, not a
measurement.",
130         "The corpus carries no atrial-fibrillation labels, so the irregularity measures are computed and described, never
validated as a screen.",
131         "The tool accepts 5.3% of a deliberately noisy corpus. Low yield is the price of not reporting a wrong rate.",
132         "A refusal is a refusal, not a finding: it says the recording could not be measured, nothing about the person.",
133         "One cohort, a small number of handsets. Nothing here characterises behaviour across skin tones, cold hands, or other
camera hardware.",
134     ],
135 }
136 (HERE / "validation.json").write_text(json.dumps(record, indent=2))
137 (HERE / "web" / "validation.json").write_text(json.dumps(record))
138
139 if __name__ == "__main__":
140     main()

```

```

1  """Choose the refusal threshold with the cost of each error visible.
2
3  Disagreeing with the annotators is not automatically wrong. What matters is
4  whether the rate we report on a trace they rejected is actually accurate. A
5  disagreement where we are right is caution we can spend; one where we are wrong
6  is a number a person might act on.
7  """
8  from __future__ import annotations
9  import csv
10 from pathlib import Path
11 import numpy as np
12 import butppg_io as io, ppg
13
14 HERE = Path(__file__).resolve().parent
15 ROOT = next(HERE.glob("data/butppg/**/*_PPG.he*")).parent.parent
16
17 ann = {}
18 with (HERE / "data" / "quality-hr-ann.csv").open(encoding="utf-8-sig") as f:
19     for r in csv.DictReader(f):
20         try: ann[r["ID"].strip()] = (int(r["Quality"]), float(r["HR"]))
21         except (ValueError, KeyError): pass
22
23 cache = []
24 for rec in sorted(p for p in ROOT.iterdir() if p.is_dir()):
25     key = rec.name
26     if key not in ann: continue
27     hea = rec / f"{key}_PPG.he*"
28     try:
29         h0 = io.read_header(hea)
30         if h0["nsig"] != 3 or h0["nsamp"] < 150: continue
31         sig, h = io.read_signal(hea)
32     except Exception: continue
33     names = [s["name"] for s in h["sigs"]]
34     ch = names.index("PPG_G") if "PPG_G" in names else 1
35     raw = sig[:, ch]
36     x = ppg.bandpass(ppg.detrend_normalise(raw), h["fs"])
37     peaks = ppg.find_beats(x, h["fs"])
38     if peaks.size < 3: continue
39     ibi = ppg.intervals_ms(peaks, h["fs"])
40     hr = 60000.0 / np.mean(ibi)
41     purity, peak_bpm = ppg.spectral_purity(x, h["fs"])
42     cache.append({"good": ann[key][0] == 1, "ref": ann[key][1], "hr": hr,
43                  "purity": purity, "peak_bpm": peak_bpm})
44
45 print(f"traces with detectable beats: {len(cache)}\n")
46 print(f"{'purity':>7} {'accept':>7} {'of which they call good':>24} {'MAE on accepted':>16} {'within 10bpm':>13}")
47 for thr in (0.80, 0.85, 0.88, 0.90, 0.92, 0.94, 0.96):
48     acc = [c for c in cache if c["purity"] >= thr
49            and abs(c["hr"] - c["peak_bpm"]) / max(c["peak_bpm"], 1) <= 0.25]
50     if not acc: continue
51     err = np.array([abs(c["hr"] - c["ref"]) for c in acc])
52     goodfrac = np.mean([c["good"] for c in acc])
53     print(f"{'thr':>7.2f} {len(acc):>7} {goodfrac*100:>23.0f}% {err.mean():>15.2f} {np.mean(err<=10)*100:>12.0f}%")

```

```

1  /**
2   * Does web/dsp.js actually agree with ppg.py?
3   *
4   * Run dsp_ref.py first; it writes dsp_fixtures.json from real BUT PPG green
5   * traces. This replays every one of them through the JavaScript and compares
6   * the whole pipeline against the Python, quantity by quantity, and prints the
7   * worst case for each. It reports what it finds rather than asserting that
8   * everything is fine.
9   *
10  * .venv/bin/python dsp_ref.py && node verify_dsp.mjs
11  */
12
13  import { readFileSync } from "node:fs";
14  import * as dsp from "./web/dsp.js";
15
16  const fixtures = JSON.parse(readFileSync(new URL("./dsp_fixtures.json", import.meta.url), "utf8"));
17  const units = JSON.parse(readFileSync(new URL("./dsp_unit_fixtures.json", import.meta.url), "utf8"));
18
19  const BARS = {
20    filtered: 1e-9, // relative to the signal's own scale
21    scalar: 1e-6, // hr, purity, mssd, cv
22    beats: 0.95, // fraction of traces with identical beat indices
23  };
24
25  /** Worst absolute difference between two same-length sequences. */
26  function maxAbsDiff(a, b) {
27    if (a.length !== b.length) return { diff: Infinity, at: -1, lenMismatch: [a.length, b.length] };
28    let diff = 0;
29    let at = -1;
30    for (let i = 0; i < a.length; i++) {
31      const d = Math.abs(a[i] - b[i]);
32      if (d > diff) { diff = d; at = i; }
33    }
34    return { diff, at };
35  }
36
37  /** Pearson correlation, as a sanity check that any divergence is noise not shape. */
38  function correlation(a, b) {
39    const n = a.length;
40    let ma = 0, mb = 0;
41    for (let i = 0; i < n; i++) { ma += a[i]; mb += b[i]; }
42    ma /= n; mb /= n;
43    let sab = 0, saa = 0, sbb = 0;
44    for (let i = 0; i < n; i++) {
45      const da = a[i] - ma;
46      const db = b[i] - mb;
47      sab += da * db; saa += da * da; sbb += db * db;
48    }
49    if (saa === 0 || sbb === 0) return saa === sbb ? 1 : 0;
50    return sab / Math.sqrt(saa * sbb);
51  }
52
53  function scale(a) {
54    let m = 0;
55    for (const v of a) m = Math.max(m, Math.abs(v));
56    return m > 0 ? m : 1;
57  }
58
59  /** Tracks the worst offender for one compared quantity. */
60  class Worst {
61    constructor(name) { this.name = name; this.n = 0; this.value = 0; this.id = "-"; this.note = ""; }
62    see(id, value, note = "") {
63      this.n++;
64      if (value > this.value) { this.value = value; this.id = id; this.note = note; }
65    }
66  }
67
68  const worst = {
69    gridAbs: new Worst("resampled grid, max abs diff"),
70    fullRel: new Worst("filtfilt over the whole trace, max abs diff / scale"),
71    winRel: new Worst("filtfilt over the chosen window, max abs diff / scale"),
72    purity: new Worst("spectral purity, abs diff"),

```

```

73   hr: new Worst("heart rate bpm, abs diff"),
74   rmssd: new Worst("rmssd ms, abs diff"),
75   cv: new Worst("cv, abs diff"),
76   sdnn: new Worst("sdnn ms, abs diff"),
77   pnn50: new Worst("pnn50, abs diff"),
78   sweepPurity: new Worst("purity, every candidate window, abs diff"),
79 };
80 // The window choice is an argmax over purities. It only means anything if the
81 // gap it decides by is far larger than the disagreement between the two
82 // implementations, so both are measured. Traces where no window reached the
83 // four-beat gate score -1.0 everywhere and are decided by order, not by
84 // arithmetic -- counted separately rather than reported as a zero margin.
85 let minMargin = Infinity;
86 let minMarginId = "-";
87 let sentinelDecided = 0;
88 let minCorrFull = 1;
89 let minCorrWin = 1;
90 let minCorrFullId = "-";
91
92 const windowSweepMismatches = [];
93 let windowsCompared = 0;
94 const beatMismatches = [];
95 const usableMismatches = [];
96 const offsetMismatches = [];
97 const reasonMismatches = [];
98 let beatsCompared = 0;
99 let beatsIdentical = 0;
100 let tiesTotal = 0;
101
102 for (const f of fixtures) {
103   tiesTotal += f.height_ties;
104
105   const grid = dsp.resampleTo30Hz(f.values, f.timestamps_ms);
106   worst.gridAbs.see(f.id, maxAbsDiff(grid, f.grid).diff);
107
108   const res = dsp.analyse(f.values, f.timestamps_ms);
109
110   if (!f.too_short) {
111     const full = dsp.bandpass(dsp.detrendNormalise(grid));
112     worst.fullRel.see(f.id, maxAbsDiff(full, f.filtered_full).diff / scale(f.filtered_full));
113     const cFull = correlation(full, f.filtered_full);
114     if (cFull < minCorrFull) { minCorrFull = cFull; minCorrFullId = f.id; }
115
116     worst.winRel.see(f.id, maxAbsDiff(res.filtered, f.filtered_window).diff / scale(f.filtered_window));
117     minCorrWin = Math.min(minCorrWin, correlation(res.filtered, f.filtered_window));
118
119     if (res.window.start_s * dsp.FS !== f.window_offset) {
120       offsetMismatches.push(`${f.id}: js ${res.window.start_s * dsp.FS} vs py ${f.window_offset}`);
121     }
122     if (res.window.seconds !== f.window_seconds) {
123       offsetMismatches.push(`${f.id}: window length js ${res.window.seconds} vs py ${f.window_seconds}`);
124     }
125     // Every candidate window, not only the one analyse kept.
126     for (const w of f.windows) {
127       windowsCompared++;
128       const x = dsp.bandpass(dsp.detrendNormalise(grid.subarray(w.off, w.off + f.window_len)));
129       const pk = dsp.findBeats(x);
130       const q = dsp.signalQuality(x, pk);
131       const [pur, bpm] = dsp.spectralPurity(x);
132       const score = "purity" in q ? q.purity : -1.0;
133       const problems = [];
134       if (pk.length !== w.peaks.length || pk.some((v, i) => v !== w.peaks[i])) {
135         problems.push(`peaks js ${JSON.stringify(pk)} vs py ${JSON.stringify(w.peaks)}`);
136       }
137       worst.sweepPurity.see(`${f.id}@${w.off}`, Math.abs(pur - w.purity));
138       if (Math.abs(pur - w.purity) > BARS.scalar) problems.push(`purity ${Math.abs(pur - w.purity)}`);
139       if (Math.abs(bpm - w.peak_bpm) > BARS.scalar) problems.push(`peak bpm ${Math.abs(bpm - w.peak_bpm)}`);
140       if (Math.abs(score - w.score) > BARS.scalar) problems.push(`score js ${score} vs py ${w.score}`);
141       if ((score === -1.0) !== (w.score === -1.0)) problems.push(`one side scored the window as unmeasurable`);
142       if (q.usable !== w.usable) problems.push(`usable js ${q.usable} vs py ${w.usable}`);
143       if ((q.reason ?? "") !== w.reason) problems.push(`reason js "${q.reason}" vs py "${w.reason}"`);
144       if (problems.length) windowSweepMismatches.push(`${f.id} off=${w.off}: ${problems.join(" ");}`);
145     }
146     const scores = f.windows.map((w) => w.score).sort((a, b) => b - a);
147     if (scores[0] === -1.0) {

```

```

148     sentinelDecided++;
149 } else if (scores.length > 1 && scores[0] - scores[1] < minMargin) {
150     minMargin = scores[0] - scores[1];
151     minMarginId = f.id;
152 }
153 } else if (res.window !== undefined) {
154     offsetMismatches.push(`${f.id}: js produced a window for a trace Python refused as too short`);
155 }
156
157 beatsCompared++;
158 const same = res.peaks.length === f.peaks.length && res.peaks.every((p, i) => p === f.peaks[i]);
159 if (same) beatsIdentical++;
160 else beatMismatches.push({ id: f.id, js: res.peaks, py: f.peaks, ties: f.height_ties });
161
162 if (res.quality.usable !== f.usable) {
163     usableMismatches.push(`${f.id}: js ${res.quality.usable} (${res.quality.reason}) vs py ${f.usable} (${f.reason})`);
164 }
165 if ((res.quality.reason ?? "") !== (f.reason ?? "")) {
166     reasonMismatches.push(`${f.id}: js "${res.quality.reason}" vs py "${f.reason}"`);
167 }
168
169 const pairs = [
170     [worst.purity, "purity" in res.quality ? res.quality.purity : null, f.purity],
171     [worst.hr, res.hr_bpm ?? null, f.hr_bpm],
172     [worst.xmssd, res.xmssd_ms ?? null, f.xmssd_ms],
173     [worst.cv, res.cv ?? null, f.cv],
174     [worst.sdn, res.sdn_ms ?? null, f.sdn_ms],
175     [worst.pnn50, res.pnn50 ?? null, f.pnn50],
176 ];
177 for (const [w, js, py] of pairs) {
178     if (js === null && py === null) continue;
179     if (js === null || py === null) { w.see(f.id, Infinity, `defined on one side only: js=${js} py=${py}`); continue; }
180     w.see(f.id, Math.abs(js - py));
181 }
182 }
183
184 // --- adversarial signals: plateaus, exact ties, degenerate spectra -----
185 const unitProblems = [];
186 const unitTieNotes = [];
187 let unitTies = 0;
188 let unitTieBreakDiffs = 0;
189 const unitWorst = {
190     detrended: new Worst("detrend_normalise, max abs diff"),
191     filt: new Worst("filtfilt, max abs diff / scale"),
192     purity: new Worst("spectral purity, abs diff"),
193     bpm: new Worst("spectral peak bpm, abs diff"),
194 };
195 for (const u of units) {
196     unitTies += u.height_ties;
197     const x = Float64Array.from(u.x);
198     unitWorst.detrended.see(u.name, maxAbsDiff(dsp.detrendNormalise(x), u.detrended).diff);
199     unitWorst.filt.see(u.name, maxAbsDiff(dsp.bandpass(x), u.filtfilt).diff / scale(u.filtfilt));
200
201     const pk = dsp.findBeats(x);
202     const eq = (a, b) => a.length === b.length && a.every((v, i) => v === b[i]);
203     const samePeaks = eq(pk, u.peaks);
204     if (!samePeaks) {
205         // The one licence to differ: peaks of exactly equal height inside one
206         // exclusion zone, which scipy resolves with an unstable sort. To count as
207         // that and not as a bug, the JS must reproduce scipy-with-a-stable-sort
208         // exactly -- same peaks, not merely plausible ones.
209         if (u.height_ties > 0 && eq(pk, u.peaks_stable)) {
210             unitTieBreakDiffs++;
211             unitTieNotes.push(`${u.name}: ${u.height_ties} exact-height ties; ` +
212                 `js matches scipy-with-stable-sort exactly, differs from scipy-as-shipped at ` +
213                 `${pk.filter((v, i) => v !== u.peaks[i]).length} of ${pk.length} beats`);
214         } else {
215             unitProblems.push(`${u.name}: peaks differ and it is NOT the tie-break ` +
216                 `(ties=${u.height_ties}, matches stable=${eq(pk, u.peaks_stable)})\n` +
217                 `js    ${JSON.stringify(pk)}\n    py    ${JSON.stringify(u.peaks)}\n` +
218                 `py-stable ${JSON.stringify(u.peaks_stable)}`);
219         }
220     } else if (!eq(pk, u.peaks_stable)) {
221         unitProblems.push(`${u.name}: matches scipy but not scipy-with-stable-sort - the harness is wrong`);
222     }

```

```

223
224 const [pur, bpm] = dsp.spectralPurity(x);
225 unitWorst.purity.see(u.name, Math.abs(pur - u.purity));
226 unitWorst.bpm.see(u.name, Math.abs(bpm - u.peak_bpm));
227
228 const ibi = dsp.intervalsMs(pk);
229 if (samePeaks) {
230   const d = maxAbsDiff(ibi, u.ibi).diff;
231   if (d !== 0) unitProblems.push(`${u.name}: intervals differ by ${d}`);
232   const irr = dsp.irregularity(ibi);
233   for (const k of Object.keys(u.irregularity)) {
234     const a = irr[k];
235     const b = u.irregularity[k];
236     if (a === undefined) { unitProblems.push(`${u.name}: irregularity missing ${k}`); continue; }
237     if (Math.abs(a - b) > BARS.scalar) unitProblems.push(`${u.name}: ${k} js ${a} vs py ${b}`);
238   }
239   const q = dsp.signalQuality(x, pk);
240   for (const k of Object.keys(u.quality)) {
241     const a = q[k];
242     const b = u.quality[k];
243     const ok = typeof b === "number" ? Math.abs(a - b) <= BARS.scalar : a === b;
244     if (!ok) unitProblems.push(`${u.name}: quality.${k} js ${JSON.stringify(a)} vs py ${JSON.stringify(b)}`);
245   }
246 }
247 }
248
249 const pad = (s, n) => String(s).padEnd(n);
250 const line = "-".repeat(78);
251
252 console.log(line);
253 console.log(`ppg.py -> web/dsp.js | ${fixtures.length} real BUT PPG traces, 30 Hz grid`);
254 console.log(`${fixtures.filter((f) => f.too_short).length} of them truncated below the minimum duration, ` +
255   `${fixtures.filter((f) => !f.too_short && f.window_len < 240).length} shorter than one window`);
256 console.log(`usable in Python: ${fixtures.filter((f) => f.usable).length} ` +
257   `exact peak-height ties found anywhere: ${tiesTotal}`);
258 console.log(line);
259
260 for (const w of Object.values(worst)) {
261   console.log(`${pad(w.name, 52)} ${w.value.toExponential(3)} (${w.n} traces, worst ${w.id})${w.note ? " " + w.note : ""}`);
262 }
263 console.log(`${pad("purity margin the window choice turns on (min)", 52)} ${minMargin.toExponential(3)} (${minMarginId})`);
264 console.log(`${pad("traces where no window passed the beat gate", 52)} ${sentinelDecided} (window picked by order, identically in
both)`);
265 console.log(`${pad("filtfilt correlation, whole trace (min)", 52)} ${minCorrFull.toFixed(15)} (${minCorrFullId})`);
266 console.log(`${pad("filtfilt correlation, chosen window (min)", 52)} ${minCorrWin.toFixed(15)}`);
267 console.log(line);
268
269 const beatRate = beatsIdentical / beatsCompared;
270 console.log(`beat indices identical : ${beatsIdentical}/${beatsCompared} (${(beatRate * 100).toFixed(2)}%)`);
271 console.log(`usable flag agrees : ${fixtures.length - usableMismatches.length}/${fixtures.length}`);
272 console.log(`refusal reason string agrees: ${fixtures.length - reasonMismatches.length}/${fixtures.length}`);
273 console.log(`chosen window offset agrees : ${fixtures.length - offsetMismatches.length}/${fixtures.length}`);
274 console.log(`every candidate window agrees: ${windowsCompared - windowSweepMismatches.length}/${windowsCompared} ` +
275   `(peaks, purity, peak bpm, score, usable, reason)`);
276
277 for (const [label, list] of [["BEAT", beatMismatches.map((m) =>
278   `${m.id}: ties=${m.ties}\n js ${JSON.stringify(m.js)}\n py ${JSON.stringify(m.py)}`)],
279   ["USABLE", usableMismatches], ["REASON", reasonMismatches], ["OFFSET", offsetMismatches],
280   ["WINDOW SWEEP", windowSweepMismatches]]) {
281   if (!list.length) continue;
282   console.log(`\n${label} MISMATCHES (${list.length})`);
283   for (const m of list.slice(0, 20)) console.log(" " + m);
284   if (list.length > 20) console.log(` ... and ${list.length - 20} more`);
285 }
286
287 console.log(line);
288 console.log(`ADVERSARIAL SIGNALS (${units.length}): plateaus, quantised levels, constants, spikes`);
289 console.log(` exact peak-height ties inside an exclusion zone: ${unitTies}`);
290 for (const w of Object.values(unitWorst)) {
291   console.log(`${pad(w.name, 44)} ${w.value.toExponential(3)} (worst ${w.id})`);
292 }
293 console.log(` signals where the beat set differs: ${unitTieBreakDiffs}/${units.length}, all attributed:`);
294 for (const m of unitTieNotes) console.log(` ${m}`);
295 if (unitProblems.length) {
296   console.log(" PROBLEMS:");

```

```

297   for (const m of unitProblems) console.log("    " + m);
298 } else {
299   console.log(" nothing unaccounted for");
300 }
301 console.log(line);
302 const checks = [
303   ["resampled grid bit-identical", worst.gridAbs.value === 0],
304   ["filtfilt (full) < ${BARS.filtered}", worst.fullRel.value < BARS.filtered],
305   ["filtfilt (window) < ${BARS.filtered}", worst.winRel.value < BARS.filtered],
306   ["beat indices identical >= ${BARS.beats * 100}%", beatRate >= BARS.beats],
307   ["heart rate < ${BARS.scalar}", worst.hr.value < BARS.scalar],
308   ["purity < ${BARS.scalar}", worst.purity.value < BARS.scalar],
309   ["mssd < ${BARS.scalar}", worst.mssd.value < BARS.scalar],
310   ["cv < ${BARS.scalar}", worst.cv.value < BARS.scalar],
311   ["usable flag 100%", usableMismatches.length === 0],
312   ["window offset 100%", offsetMismatches.length === 0],
313   ["every candidate window 100%", windowSweepMismatches.length === 0],
314   ["adversarial signals clean", unitProblems.length === 0],
315 ];
316 let failed = 0;
317 for (const [name, ok] of checks) {
318   if (!ok) failed++;
319   console.log(` ${ok ? "PASS" : "FAIL"} ${name}`);
320 }
321 console.log(line);
322 console.log(failed === 0 ? "ALL BARS MET" : `${failed} BAR(S) NOT MET`);
323 process.exit(failed === 0 ? 0 : 1);

```

```

1  /**
2   * Pulse rhythm from a photoplethysmogram, in the browser.
3   *
4   * A line-for-line port of ppg.py. The Python is the source of truth; this file
5   * exists so the phone can do the arithmetic without shipping the frames
6   * anywhere. verify_dsp.mjs checks the two against each other on real BUT PPG
7   * traces, because a port that quietly drifts is worse than no port.
8   *
9   * Everything here runs at a fixed 30 Hz. Camera frame rates wander -- 28 Hz on
10  * a cold phone, 31 on a warm one, and neither is uniform -- so the caller
11  * resamples onto a 30 Hz grid first and the filter never has to be redesigned.
12  */
13
14  export const HR_MIN_BPM = 30.0;
15  export const HR_MAX_BPM = 200.0;
16  export const PURITY_MIN = 0.85;
17  export const AGREEMENT_TOL = 0.05;
18  export const LOW_HZ = HR_MIN_BPM / 60.0;
19  export const HIGH_HZ = HR_MAX_BPM / 60.0;
20  export const MIN_SECONDS = 4.0;
21  export const WINDOW_SECONDS = 8.0;
22  export const WINDOW_STEP_SECONDS = 0.5;
23
24  /** The one sample rate this module is designed for. */
25  export const FS = 30.0;
26
27  // scipy.signal.butter(3, [0.5/15, 3.333333333333335/15], btype="band") at
28  // fs = 30. Designing a Butterworth in JS would mean porting bilinear
29  // transforms and complex pole placement for a filter that never changes, so
30  // the coefficients are lifted verbatim -- shortest round-trip decimal, which
31  // is exact for float64.
32  export const B = [
33    0.015641668161113648, 0.0, -0.04692500448334094, 0.0,
34    0.04692500448334094, 0.0, -0.015641668161113648,
35  ];
36  export const A = [
37    1.0, -4.646768904846301, 9.153515799958402, -9.833575204677665,
38    6.0972389352023795, -2.0695294630527683, 0.2993485646941389,
39  ];
40  // scipy.signal.lfilter_zi(B, A): the delay state that holds a constant input
41  // at a constant output. filtfilt scales it by the first sample of each pass so
42  // the filter starts already settled instead of ringing for the first second.
43  export const ZI = [
44    -0.015641668161113648, -0.015641668161113648, 0.031283336322227295,
45    0.031283336322227295, -0.015641668161113648, -0.015641668161113648,
46  ];
47  // filtfilt's default: 3 * max(len(a), len(b)).
48  const PADLEN = 3 * Math.max(A.length, B.length);
49
50  /**
51   * numpy's pairwise summation, reproduced.
52   *
53   * np.mean and np.std do not add left to right; they use eight accumulators
54   * under 128 elements and recurse above it. Summing naively here puts the two
55   * languages ~1e-16 apart, which is harmless for a heart rate but can tip a
56   * near-tied peak and change the beat indices. Cheap insurance.
57   */
58  function pairwiseSum(a, off, n) {
59    if (n < 8) {
60      let res = 0.0;
61      for (let i = 0; i < n; i++) res += a[off + i];
62      return res;
63    }
64    if (n <= 128) {
65      const r = [a[off], a[off + 1], a[off + 2], a[off + 3],
66        a[off + 4], a[off + 5], a[off + 6], a[off + 7]];
67      let i = 8;
68      for (; i < n - (n % 8); i += 8) {
69        r[0] += a[off + i]; r[1] += a[off + i + 1];
70        r[2] += a[off + i + 2]; r[3] += a[off + i + 3];
71        r[4] += a[off + i + 4]; r[5] += a[off + i + 5];
72        r[6] += a[off + i + 6]; r[7] += a[off + i + 7];

```

```

73     }
74     let res = ((r[0] + r[1]) + (r[2] + r[3])) + ((r[4] + r[5]) + (r[6] + r[7]));
75     for (; i < n; i++) res += a[off + i];
76     return res;
77     }
78     let n2 = (n / 2) | 0;
79     n2 -= n2 % 8;
80     return pairwiseSum(a, off, n2) + pairwiseSum(a, off + n2, n - n2);
81 }
82
83 function mean(x) {
84     return pairwiseSum(x, 0, x.length) / x.length;
85 }
86
87 /** Population standard deviation, ddof=0, as np.std computes it. */
88 function std(x) {
89     const m = pairwiseSum(x, 0, x.length) / x.length;
90     const sq = new Float64Array(x.length);
91     for (let i = 0; i < x.length; i++) {
92         const d = x[i] - m;
93         sq[i] = d * d;
94     }
95     return Math.sqrt(pairwiseSum(sq, 0, sq.length) / sq.length);
96 }
97
98 /**
99  * Linear resample of camera samples onto a uniform 30 Hz grid.
100  *
101  * The grid starts at the first frame's timestamp and runs to the last one that
102  * still falls inside the recording, so no sample is invented past the end.
103  * Values outside the input range clamp to the endpoints, matching np.interp.
104  */
105 export function resampleTo30Hz(values, timestampsMs) {
106     const n = Math.min(values.length, timestampsMs.length);
107     if (n === 0) return new Float64Array(0);
108     if (n === 1) return Float64Array.of(values[0]);
109     const t0 = timestampsMs[0];
110     const span = timestampsMs[n - 1] - t0;
111     const dt = 1000.0 / FS;
112     const count = Math.floor(span / dt) + 1;
113     const out = new Float64Array(count);
114     let j = 0;
115     for (let i = 0; i < count; i++) {
116         const t = t0 + i * dt;
117         while (j + 2 < n && timestampsMs[j + 1] < t) j++;
118         const ta = timestampsMs[j];
119         const tb = timestampsMs[j + 1];
120         if (t <= ta) { out[i] = values[j]; continue; }
121         if (t >= tb) { out[i] = values[j + 1]; continue; }
122         const w = (t - ta) / (tb - ta);
123         out[i] = values[j] + w * (values[j + 1] - values[j]);
124     }
125     return out;
126 }
127
128 /**
129  * scipy.signal.lfilter for a fixed-length IIR with a given initial state.
130  *
131  * Transposed direct form II, with the same operation grouping scipy's C loop
132  * uses so the rounding lands identically.
133  */
134 function lfilter(x, zi) {
135     const n = x.length;
136     const N = A.length;
137     const y = new Float64Array(n);
138     const z = zi.slice();
139     for (let k = 0; k < n; k++) {
140         const xk = x[k];
141         const yk = B[0] * xk + z[0];
142         y[k] = yk;
143         for (let i = 0; i < N - 2; i++) {
144             z[i] = B[i + 1] * xk + z[i + 1] - A[i + 1] * yk;
145         }
146         z[N - 2] = B[N - 1] * xk - A[N - 1] * yk;
147     }

```

```

148   return y;
149 }
150
151 /**
152  * scipy.signal.filtfilt with its defaults: odd padding, padlen 3*ntaps, and
153  * the lfilter_zi trick on both passes.
154  *
155  * The odd extension reflects the signal through its own endpoint -- 2*x[0]
156  * minus the mirrored run -- so the filter sees a continuous slope instead of a
157  * step at the edges. Dropping any of these three pieces still produces a
158  * plausible-looking filtered trace, which is exactly why it has to be checked.
159  */
160 export function filtfilt(x) {
161   const n = x.length;
162   if (n <= PADLEN) {
163     throw new Error(`the length of the input vector must be greater than ${PADLEN}`);
164   }
165   const ext = new Float64Array(n + 2 * PADLEN);
166   const first = x[0];
167   const last = x[n - 1];
168   for (let i = 0; i < PADLEN; i++) ext[i] = 2 * first - x[PADLEN - i];
169   ext.set(x, PADLEN);
170   for (let i = 0; i < PADLEN; i++) ext[n + PADLEN + i] = 2 * last - x[n - 2 - i];
171
172   const ziF = ZI.map((v) => v * ext[0]);
173   const fwd = lfilter(ext, ziF);
174
175   const rev = new Float64Array(fwd.length);
176   for (let i = 0; i < fwd.length; i++) rev[i] = fwd[fwd.length - 1 - i];
177   const ziB = ZI.map((v) => v * rev[0]);
178   const back = lfilter(rev, ziB);
179
180   const out = new Float64Array(n);
181   for (let i = 0; i < n; i++) out[i] = back[back.length - 1 - PADLEN - i];
182   return out;
183 }
184
185 export function bandpass(x, fs = FS) {
186   if (fs !== FS) throw new Error(`coefficients are fixed at ${FS} Hz, got ${fs}`);
187   return filtfilt(x);
188 }
189
190 export function detrendNormalise(x) {
191   const out = new Float64Array(x.length);
192   const m = mean(x);
193   for (let i = 0; i < x.length; i++) out[i] = x[i] - m;
194   const s = std(out);
195   if (s > 0) for (let i = 0; i < out.length; i++) out[i] /= s;
196   return out;
197 }
198
199 /** scipy's _local_maxima_1d: midpoints of flat plateaus, not just strict peaks. */
200 function localMaxima1d(x) {
201   const peaks = [];
202   const iMax = x.length - 1;
203   let i = 1;
204   while (i < iMax) {
205     if (x[i - 1] < x[i]) {
206       let ahead = i + 1;
207       while (ahead < iMax && x[ahead] === x[i]) ahead++;
208       if (x[ahead] < x[i]) {
209         peaks.push((i + (ahead - 1)) >> 1);
210         i = ahead;
211       }
212     }
213     i++;
214   }
215   return peaks;
216 }
217
218 /**
219  * scipy's _select_by_peak_distance: keep the tallest, then suppress everything
220  * within 'distance' of it, working down the heights.
221  *
222  * scipy sorts with np.argsort's default quicksort, so peaks of exactly equal

```

```

223 * height inside one another's exclusion zone are resolved arbitrarily -- and
224 * on numpy 2 that call can dispatch to an AVX-512 kernel, so scipy's own order
225 * is not fixed across machines. There is nothing stable to copy, so this sorts
226 * stably and the harness proves the difference is confined to that: on
227 * quantised signals the JS reproduces scipy-with-a-stable-sort beat for beat,
228 * and on 1,570 windows of real filtered PPG no such tie occurs at all.
229 */
230 function selectByPeakDistance(peaks, priority, distance) {
231   const size = peaks.length;
232   const d = Math.ceil(distance);
233   const keep = new Uint8Array(size).fill(1);
234   const order = Array.from({ length: size }, (_, i) => i);
235   order.sort((p, q) => priority[p] - priority[q]);
236   for (let i = size - 1; i >= 0; i--) {
237     const j = order[i];
238     if (keep[j] === 0) continue;
239     let k = j - 1;
240     while (k >= 0 && peaks[j] - peaks[k] < d) { keep[k] = 0; k--; }
241     k = j + 1;
242     while (k < size && peaks[k] - peaks[j] < d) { keep[k] = 0; k++; }
243   }
244   return keep;
245 }
246
247 /**
248 * scipy's _peak_prominences with wlen unset: walk outwards from each peak
249 * until the signal rises above it, and take the higher of the two valleys.
250 */
251 function peakProminences(x, peaks) {
252   const out = new Float64Array(peaks.length);
253   const iMax = x.length - 1;
254   for (let p = 0; p < peaks.length; p++) {
255     const peak = peaks[p];
256     let leftMin = x[peak];
257     let i = peak;
258     while (i >= 0 && x[i] <= x[peak]) {
259       if (x[i] < leftMin) leftMin = x[i];
260       i--;
261     }
262     let rightMin = x[peak];
263     i = peak;
264     while (i <= iMax && x[i] <= x[peak]) {
265       if (x[i] < rightMin) rightMin = x[i];
266       i++;
267     }
268     out[p] = x[peak] - Math.max(leftMin, rightMin);
269   }
270   return out;
271 }
272
273 /**
274 * Beat sample indices.
275 *
276 * Minimum spacing comes from the maximum plausible rate, so one beat cannot
277 * be counted twice. Prominence is relative to the signal's own spread, so it
278 * adapts to a weak trace instead of needing a magic constant.
279 *
280 * scipy applies the distance rule before the prominence rule, and the two are
281 * not commutative: a tall peak can suppress a neighbour and then be discarded
282 * itself, leaving a gap. Swapping them is a silent behaviour change.
283 */
284 export function findBeats(x, fs = FS) {
285   const minGap = Math.trunc(fs * 60.0 / HR_MAX_BPM);
286   const prom = 0.3 * std(x);
287   let peaks = localMaxima1d(x);
288   const keepD = selectByPeakDistance(peaks, peaks.map((i) => x[i]), Math.max(minGap, 1));
289   peaks = peaks.filter((_, i) => keepD[i] === 1);
290   const proms = peakProminences(x, peaks);
291   return peaks.filter((_, i) => prom <= proms[i]);
292 }
293
294 export function intervalsMs(peaks, fs = FS) {
295   if (peaks.length < 2) return new Float64Array(0);
296   const out = new Float64Array(peaks.length - 1);
297   for (let i = 0; i < out.length; i++) out[i] = (peaks[i + 1] - peaks[i]) / fs * 1000.0;

```

```

298     return out;
299 }
300
301 /**
302  * Standard beat-to-beat variability measures.
303  *
304  * rmssd and pnn50 are the usual descriptors of how much consecutive beats
305  * differ. Atrial fibrillation is irregularly irregular, so both run high;
306  * a steady rhythm keeps them low. These are descriptions of the trace, not
307  * a diagnosis of the heart.
308  */
309 export function irregularity(ibi) {
310     if (ibi.length < 3) {
311         return { n_beats: ibi.length ? ibi.length + 1 : 0 };
312     }
313     const diffs = new Float64Array(ibi.length - 1);
314     for (let i = 0; i < diffs.length; i++) diffs[i] = ibi[i + 1] - ibi[i];
315     const sq = new Float64Array(diffs.length);
316     for (let i = 0; i < diffs.length; i++) sq[i] = diffs[i] * diffs[i];
317     const rmssd = Math.sqrt(mean(sq));
318     const over = new Float64Array(diffs.length);
319     for (let i = 0; i < diffs.length; i++) over[i] = Math.abs(diffs[i]) > 50.0 ? 1 : 0;
320     const meanIbi = mean(ibi);
321     const sd = std(ibi);
322     return {
323         n_beats: ibi.length + 1,
324         mean_ibi_ms: meanIbi,
325         hr_bpm: meanIbi > 0 ? 60000.0 / meanIbi : 0.0,
326         sdn_ms: sd,
327         rmssd_ms: rmssd,
328         pnn50: mean(over),
329         cv: meanIbi > 0 ? sd / meanIbi : 0.0,
330     };
331 }
332
333 // np.hanning(n) and the rfft twiddles depend only on the window length, which
334 // is constant across every candidate window of a recording.
335 const fftCache = new Map();
336
337 function fftTables(n) {
338     let t = fftCache.get(n);
339     if (t) return t;
340     const win = new Float64Array(n);
341     if (n === 1) {
342         win[0] = 1.0;
343     } else {
344         for (let i = 0; i < n; i++) win[i] = 0.5 + 0.5 * Math.cos(Math.PI * (1 - n + 2 * i) / (n - 1));
345     }
346     const half = (n >> 1) + 1;
347     // np.fft.rfftfreq(n, 1/fs) to the bit: it forms 1/(n*d) once and multiplies,
348     // which is not the same float as k*fs/n. The band edges are compared against
349     // these, so a one-ulp difference could include or drop a bin.
350     const val = 1.0 / (n * (1.0 / FS));
351     const freqs = new Float64Array(half);
352     for (let k = 0; k < half; k++) freqs[k] = k * val;
353     // One period of cos/sin at the fundamental; every bin's twiddle is a stride
354     // through it, which keeps the angles exact multiples of 2*pi/n.
355     const cos = new Float64Array(n);
356     const sin = new Float64Array(n);
357     for (let i = 0; i < n; i++) {
358         const ang = -2.0 * Math.PI * i / n;
359         cos[i] = Math.cos(ang);
360         sin[i] = Math.sin(ang);
361     }
362     t = { win, half, freqs, cos, sin };
363     fftCache.set(n, t);
364     return t;
365 }
366
367 /**
368  * How much of the in-band power sits in one peak, and where that peak is.
369  *
370  * A real pulse is close to periodic, so its spectrum has one dominant lobe in
371  * the heart-rate band. A trace dominated by movement smears across the band.
372  * This turns out to separate measurable from unmeasurable far better than

```

```

373 * anything measured in the time domain.
374 *
375 * A direct  $O(n^2)$  transform rather than an FFT: n is at most 240 here, the
376 * whole sweep costs under a millisecond, and there is no radix bookkeeping to
377 * get wrong.
378 */
379 export function spectralPurity(x, fs = FS) {
380   const n = x.length;
381   const { win, half, freqs, cos, sin } = fftTables(n);
382   const xw = new Float64Array(n);
383   for (let i = 0; i < n; i++) xw[i] = x[i] * win[i];
384
385   // The in-band bins are contiguous, and they are the only ones ever read, so
386   // only those get transformed.
387   let kLo = -1;
388   let kHi = -2;
389   for (let k = 0; k < half; k++) {
390     if (freqs[k] >= LOW_HZ && freqs[k] <= HIGH_HZ) {
391       if (kLo < 0) kLo = k;
392       kHi = k;
393     }
394   }
395   if (kLo < 0) return [0.0, 0.0];
396
397   const bp = new Float64Array(kHi - kLo + 1);
398   for (let k = kLo; k <= kHi; k++) {
399     let re = 0.0;
400     let im = 0.0;
401     for (let i = 0; i < n; i++) {
402       const idx = (i * k) % n;
403       re += xw[i] * cos[idx];
404       im += xw[i] * sin[idx];
405     }
406     const mag = Math.hypot(re, im);
407     bp[k - kLo] = mag * mag;
408   }
409
410   const total = pairwiseSum(bp, 0, bp.length);
411   if (!(total > 0)) return [0.0, 0.0];
412
413   let kMax = 0;
414   for (let i = 1; i < bp.length; i++) if (bp[i] > bp[kMax]) kMax = i;
415   const lo = Math.max(kMax - 1, 0);
416   const hi = Math.min(kMax + 2, bp.length);
417   return [pairwiseSum(bp, lo, hi - lo) / total, freqs[kLo + kMax] * 60.0];
418 }
419
420 /**
421 * Can this trace be measured at all?
422 *
423 * Most smartphone PPG recordings cannot. In BUT PPG, 3,058 of 3,888 records
424 * are annotated poor quality. Refusing is the common correct answer, so the
425 * tool has to be good at it rather than treat it as an edge case.
426 */
427 export function signalQuality(x, peaks, fs = FS) {
428   // Four peaks, not three: three peaks give two intervals, and the spread of
429   // two numbers says nothing about rhythm. A trace that cannot support the
430   // measurement should not pass the gate that guards it.
431   if (peaks.length < 4) {
432     return { usable: false, reason: "too few beats detected", beats: peaks.length };
433   }
434   const ibi = intervalsMs(peaks, fs);
435   const hr = 60000.0 / mean(ibi);
436   if (!(HR_MIN_BPM <= hr && hr <= HR_MAX_BPM)) {
437     return { usable: false, reason: `implied rate ${fmt0(hr)} bpm is outside 30-200`, beats: peaks.length };
438   }
439   const [purity, peakBpm] = spectralPurity(x, fs);
440   if (purity < PURITY_MIN) {
441     return { usable: false, beats: peaks.length, purity,
442       reason: "no single dominant pulse frequency; likely movement or poor contact" };
443   }
444   if (peakBpm > 0 && Math.abs(hr - peakBpm) / peakBpm > AGREEMENT_TOL) {
445     return { usable: false, beats: peaks.length, purity,
446       reason: `counted ${fmt0(hr)} bpm but the spectrum says ${fmt0(peakBpm)} bpm` };
447   }

```

```

448     return { usable: true, reason: "", beats: peaks.length, purity };
449 }
450
451 /**
452  * Python's "%.0f", which breaks an exact tie to the even neighbour.
453  *
454  * Math.round breaks it toward +Infinity instead, so the two disagree whenever
455  * that answer is odd, and the other candidate is r-1 in both signs. Python
456  * also keeps the sign of a negative value that rounds to zero. Only refusal
457  * messages reach this, but the harness compares those word for word.
458  */
459 function fmt0(v) {
460     const r = Math.round(v);
461     const n = Math.abs(v - Math.trunc(v)) === 0.5 && r % 2 !== 0 ? r - 1 : r;
462     return n === 0 && (v < 0 || Object.is(v, -0)) ? "-0" : String(n);
463 }
464
465 /**
466  * Python's "%.1f".
467  *
468  * The only values sitting exactly halfway between two tenths are the odd
469  * quarters -- 0.25, 0.75, 1.25 -- since a half-tenth is a dyadic rational
470  * nowhere else. Python rounds those to the even tenth, toFixed rounds away
471  * from zero. Multiplying by four is exact, so the test for the case is too.
472  */
473 function fmt1(v) {
474     const q = v * 4;
475     if (Number.isInteger(q) && Math.abs(q) % 2 === 1) {
476         const lo = Math.floor(v * 10);
477         return ((lo % 2 === 0 ? lo : lo + 1) / 10).toFixed(1);
478     }
479     return v.toFixed(1);
480 }
481
482 /**
483  * Run the whole pipeline over a recording already on the 30 Hz grid.
484  *
485  * Every candidate window is scored; the calmest one is the one we report.
486  * Picking by purity rather than by the rate it produces keeps the choice
487  * independent of the answer, so this cannot quietly select for a number.
488  */
489 export function analyseGrid(values, fs = FS) {
490     const raw = values instanceof Float64Array ? values : Float64Array.from(values);
491     const total = raw.length;
492     if (total < Math.max(Math.trunc(MIN_SECONDS * fs), 25)) {
493         return { fs, peaks: [], quality: {
494             usable: false,
495             reason: `recording is ${fmt1(total / fs)} s; needs at least ${fmt0(MIN_SECONDS)} s`,
496             beats: 0 } };
497     }
498     let n = Math.trunc(WINDOW_SECONDS * fs);
499     let offsets;
500     if (total <= n) {
501         offsets = [0];
502         n = total;
503     } else {
504         const step = Math.max(Math.trunc(WINDOW_STEP_SECONDS * fs), 1);
505         offsets = [];
506         for (let o = 0; o <= total - n; o += step) offsets.push(o);
507     }
508
509     let best = null;
510     for (const off of offsets) {
511         const x = bandpass(detrendNormalise(raw.subarray(off, off + n)), fs);
512         const peaks = findBeats(x, fs);
513         const q = signalQuality(x, peaks, fs);
514         const score = "purity" in q ? q.purity : -1.0;
515         if (best === null || score > best.score) best = { score, off, x, peaks, q };
516     }
517
518     const out = {
519         fs,
520         quality: best.q,
521         peaks: best.peaks,
522         filtered: best.x,

```

```

523     window: { start_s: best.off / fs, seconds: n / fs, of_seconds: total / fs },
524   };
525   if (best.q.usable) Object.assign(out, irregularity(intervalsMs(best.peaks, fs)));
526   return out;
527 }
528
529 /**
530  * Camera samples in, rhythm out. The timestamps are the per-frame capture
531  * times in milliseconds; they are the only reason this can run on hardware
532  * whose frame rate is a suggestion rather than a promise.
533  */
534 export function analyse(values, timestampsMs) {
535   return analyseGrid(resampleTo30Hz(values, timestampsMs), FS);
536 }

```

```

1  /* Wiring for the measurement console.
2  *
3  * Everything numeric happens in dsp.js, which is a line-for-line port of the
4  * Python in ppg.py. This file only decides what to draw. The split matters:
5  * the same arithmetic that produced the published validation numbers is the
6  * arithmetic running in the browser, and verify_dsp.mjs proves it. */
7
8  import {
9    analyse, resampleTo30Hz, detrendNormalise, bandpass,
10    PURITY_MIN, AGREEMENT_TOL, WINDOW_SECONDS,
11    HR_MIN_BPM, HR_MAX_BPM,
12  } from "./dsp.js";
13
14  const S = {
15    samples: [],
16    current: null,    // the analysed result being shown
17    trace: null,      // { values, timestamps, label, ecg_bpm, expect }
18    validation: null,
19    view: "measure",
20    recording: false,
21  };
22
23  const $ = id => document.getElementById(id);
24  const el = (tag, cls, txt) => {
25    const n = document.createElement(tag);
26    if (cls) n.className = cls;
27    if (txt != null) n.textContent = txt;
28    return n;
29  };
30
31  /* ----- boot */
32
33  async function boot() {
34    const [samples, validation] = await Promise.all([
35      fetch("samples/samples.json").then(r => r.json()),
36      fetch("validation.json").then(r => r.json()).catch(() => null),
37    ]);
38    S.samples = samples;
39    S.validation = validation;
40
41    const sel = $("sample");
42    samples.forEach((s, i) => {
43      const o = el("option", null, `${s.title} · ${s.id}`);
44      o.value = String(i);
45      sel.appendChild(o);
46    });
47    sel.onchange = () => loadSample(Number(sel.value));
48
49    document.querySelectorAll(".tabs button").forEach(b => {
50      b.onclick = () => show(b.dataset.view);
51    });
52    $("replay").onclick = () => { if (S.trace) measure(S.trace); };
53    $("camera").onclick = startCamera;
54
55    loadSample(0);
56  }
57
58  function loadSample(i) {
59    const s = S.samples[i];
60    // The recordings are uniformly sampled at 30 Hz, so their timestamps are
61    // synthetic here. A camera's are not, which is why the pipeline resamples.
62    const timestamps = s.values.map((_, k) => (k * 1000) / s.fs);
63    measure({
64      values: s.values, timestamps, label: `${s.title} · record ${s.id}`,
65      note: s.note, ecg_bpm: s.ecg_bpm, expect: s.expect, id: s.id,
66    });
67  }
68
69  function measure(trace) {
70    S.trace = trace;
71    S.current = analyse(trace.values, trace.timestamps);
72    $("srcpill").textContent = trace.fromCamera ? "Your camera" : "Recorded trace";

```

```

73   $("toolnote").textContent = trace.note || "";
74   const q = S.current.quality;
75   $("statusright").textContent = q.usable
76     ? `${S.current.hr_bpm.toFixed(1)} bpm · purity ${q.purity.toFixed(3)} · ${q.beats} beats`
77     : `refused · ${q.reason}`;
78   show("measure");
79 }
80
81 /* ----- views */
82
83 function show(view) {
84   S.view = view;
85   document.querySelectorAll(".tabs button")
86     .forEach(b => b.classList.toggle("on", b.dataset.view === view));
87   const main = $("main");
88   main.innerHTML = "";
89   ({ measure: viewMeasure, method: viewMethod,
90     evidence: viewEvidence, limits: viewLimits }[view])(main);
91 }
92
93 function viewMeasure(main) {
94   const r = S.current, q = r.quality, t = S.trace;
95
96   const card = el("div", "card");
97   const head = el("div", "card-head");
98   const h = el("div");
99   h.appendChild(el("h2", null, q.usable ? "Measured" : "Not measured"));
100  h.appendChild(el("p", null, q.usable
101    ? "Two independent estimates of the rate agreed and the trace held a single dominant pulse frequency, so this reading is
reported."
102    : `Nothing is reported for this recording. ${cap(q.reason)}.`));
103  head.appendChild(h);
104  const hr = el("div", "head-right");
105  const v = el("span", `verdict ${q.usable ? "ok" : "no"}`);
106  v.appendChild(el("i", null, q.usable ? "●" : "○"));
107  v.appendChild(el("span", null, q.usable ? "Accepted" : "Refused"));
108  hr.appendChild(v);
109  head.appendChild(hr);
110  card.appendChild(head);
111
112  const stats = el("div", "stats");
113  stats.appendChild(stat("Heart rate", q.usable ? r.hr_bpm.toFixed(1) : "-", q.usable ? "bpm" : "", !q.usable));
114  stats.appendChild(stat("Beats found", String(q.beats), "", false));
115  stats.appendChild(stat("Signal purity", (q.purity ?? 0).toFixed(3), `of ${PURITY_MIN}`, false));
116  stats.appendChild(stat("Window measured",
117    r.window ? `${r.window.start_s.toFixed(1)}-${(r.window.start_s + r.window.seconds).toFixed(1)}` : "-",
118    r.window ? `s of ${r.window.of_seconds.toFixed(0)}` : "", false));
119  card.appendChild(stats);
120
121  const plot = el("div", "plot");
122  const cv = el("canvas");
123  plot.appendChild(cv);
124  card.appendChild(plot);
125  const lg = el("div", "plot-legend");
126  lg.appendChild(legend("fill", "window measured"));
127  lg.appendChild(legend("", "filtered pulse signal"));
128  lg.appendChild(legend("dot", "detected beat"));
129  card.appendChild(lg);
130  main.appendChild(card);
131
132  requestAnimationFrame(() => drawTrace(cv, r, t));
133
134  main.appendChild(checksCard(r, t));
135  if (t.ecg_bpm !== null) main.appendChild(truthCard(r, t));
136 }
137
138 function stat(k, v, unit, dim) {
139   const d = el("div", "stat");
140   d.appendChild(el("div", "stat-k", k));
141   const val = el("div", `stat-v${dim ? " dim" : ""}`, v);
142   if (unit) val.appendChild(el("span", "stat-u", unit));
143   d.appendChild(val);
144   return d;
145 }
146

```

```

147 function legend(kind, text) {
148   const s = el("span");
149   s.appendChild(el("i", `sw ${kind}`.trim()));
150   s.appendChild(el("span", null, text));
151   return s;
152 }
153
154 function checksCard(r, t) {
155   const q = r.quality;
156   const card = el("div", "card");
157   const head = el("div", "card-head");
158   const h = el("div");
159   h.appendChild(el("h2", null, "Why this recording was accepted or refused"));
160   h.appendChild(el("p", null, "Four checks, in order. The first that fails stops the measurement – the tool reports nothing rather
than a number it cannot stand behind."));
161   head.appendChild(h);
162   card.appendChild(head);
163
164   const rows = el("div", "rows");
165   const rh = el("div", "row-head");
166   rh.appendChild(el("div", null, "Check"));
167   rh.appendChild(el("div", null, "What it rules out"));
168   rh.appendChild(el("div", null, "Result"));
169   rows.appendChild(rh);
170
171   const reason = q.reason || "";
172   const beatsOk = q.beats >= 4;
173   const rateOk = beatsOk && !reason.startsWith("implied rate");
174   const purityOk = rateOk && q.purity != null && q.purity >= PURITY_MIN;
175   const agreeOk = purityOk && !reason.startsWith("counted");
176
177   rows.appendChild(check("At least four beats", "Three beats give two intervals, and two numbers say nothing about rhythm.",
178     beatsOk ? "pass" : "fail", `${q.beats} found`));
179   rows.appendChild(check("Rate is physiological", `Outside ${HR_MIN_BPM}–${HR_MAX_BPM} bpm it is noise, not a pulse.`,
180     !beatsOk ? "skip" : rateOk ? "pass" : "fail", !beatsOk ? "—" : rateOk ? "in range" : "out of range"));
181   rows.appendChild(check("One dominant pulse frequency", "A moving finger smears power across the band instead of concentrating
it.",
182     !rateOk ? "skip" : purityOk ? "pass" : "fail",
183     q.purity != null ? q.purity.toFixed(3) : "—"));
184   rows.appendChild(check("Two estimates agree",
185     `Counting beats and reading the spectrum are independent. Requiring them within ${AGREEMENT_TOL * 100}.toFixed(0)}% is what
catches half-rate errors from missed beats.`,
186     !purityOk ? "skip" : agreeOk ? "pass" : "fail",
187     !purityOk ? "—" : agreeOk ? "agree" : "disagree"));
188
189   card.appendChild(rows);
190   return card;
191 }
192
193 function check(name, why, state, value) {
194   const row = el("div", "row");
195   row.appendChild(el("div", "row-name", name));
196   row.appendChild(el("div", "row-why", why));
197   const t = el("div");
198   t.appendChild(el("span", `tag ${state}`, value));
199   row.appendChild(t);
200   return row;
201 }
202
203 function truthCard(r, t) {
204   const q = r.quality;
205   const card = el("div", "card");
206   const head = el("div", "card-head");
207   const h = el("div");
208   h.appendChild(el("h2", null, "Against the ECG recorded at the same moment"));
209   const ours = q.usable ? r.hr_bpm : null;
210   const diff = ours == null ? null : Math.abs(ours - t.ecg_bpm);
211   h.appendChild(el("p", null, ours == null
212     ? `This recording was refused. The ECG running alongside it gives ${t.ecg_bpm.toFixed(1)} bpm – so the refusal cost a reading
that was there to be had, or avoided one that would have been wrong. Both happen, and the checks cannot tell which in advance.`
213     : `The tool reports ${ours.toFixed(1)} bpm. The R-peaks on the simultaneous ECG give ${t.ecg_bpm.toFixed(1)} bpm, a difference
of ${diff.toFixed(1)} bpm.`));
214   head.appendChild(h);
215   card.appendChild(head);
216

```

```

217 if (t.expect) {
218   const rows = el("div", "rows");
219   const rh = el("div", "row-head");
220   rh.appendChild(el("div", null, "Quantity"));
221   rh.appendChild(el("div", null, "Python, on the same recording"));
222   rh.appendChild(el("div", null, "This browser"));
223   rows.appendChild(rh);
224   const pairs = [
225     ["Accepted", String(t.expect.usable), String(q.usable)],
226     ["Heart rate", t.expect.bpm == null ? "-" : `${t.expect.bpm.toFixed(2)} bpm`,
227       q.usable ? `${r.hr_bpm.toFixed(2)} bpm` : "-"],
228     ["Signal purity", t.expect.purity.toFixed(4), (q.purity ?? 0).toFixed(4)],
229   ];
230   for (const [k, a, b] of pairs) {
231     const row = el("div", "row");
232     row.appendChild(el("div", "row-name", k));
233     row.appendChild(el("div", "row-why", a));
234     const tg = el("div");
235     tg.appendChild(el("span", `tag ${a} === b ? "pass" : "fail"`, b));
236     row.appendChild(tg);
237     rows.appendChild(row);
238   }
239   card.appendChild(rows);
240 }
241 return card;
242 }
243
244 /* ----- plot */
245
246 function drawTrace(canvas, r, t) {
247   const dpr = window.devicePixelRatio || 1;
248   const w = canvas.clientWidth, h = canvas.clientHeight;
249   canvas.width = w * dpr; canvas.height = h * dpr;
250   const c = canvas.getContext("2d");
251   c.scale(dpr, dpr);
252   c.clearRect(0, 0, w, h);
253
254   // `r.filtered` covers only the window that was measured. The chart shows the
255   // whole recording so the chosen window can be seen in context, so the full
256   // trace is filtered again here purely for display.
257   let y;
258   try {
259     y = bandpass(detrendNormalise(resampleTo30Hz(t.values, t.timestamps)));
260   } catch {
261     y = t.values;
262   }
263   if (!y || !y.length) return;
264   const n = y.length;
265   let lo = Infinity, hi = -Infinity;
266   for (const v of y) { if (v < lo) lo = v; if (v > hi) hi = v; }
267   const pad = (hi - lo) * 0.12 || 1;
268   lo -= pad; hi += pad;
269   const X = i => (i / (n - 1)) * (w - 2) + 1;
270   const Y = v => h - 16 - ((v - lo) / (hi - lo)) * (h - 30);
271
272   // the stretch actually measured, behind everything else
273   if (r.window) {
274     const fs = r.fs || 30;
275     const a = X(r.window.start_s * fs);
276     const b = X((r.window.start_s + r.window.seconds) * fs);
277     c.fillStyle = "#ddd9f8";
278     c.fillRect(a, 6, Math.max(b - a, 1), h - 22);
279   }
280
281   c.strokeStyle = "#e5e7eb";
282   c.lineWidth = 1;
283   c.beginPath(); c.moveTo(0, h - 16); c.lineTo(w, h - 16); c.stroke();
284
285   c.strokeStyle = "#4338ca";
286   c.lineWidth = 1.6;
287   c.lineJoin = "round";
288   c.beginPath();
289   for (let i = 0; i < n; i++) {
290     const px = X(i), py = Y(y[i]);
291     i ? c.lineTo(px, py) : c.moveTo(px, py);

```

```

292 }
293 c.stroke();
294
295 // beat indices are relative to the window, so shift them onto the full trace
296 const off = r.window ? Math.round(r.window.start_s * (r.fs || 30)) : 0;
297 for (const p of (r.peaks || [])) {
298   const idx = p + off;
299   if (idx < 0 || idx >= n) continue;
300   c.fillStyle = "#4338ca";
301   c.beginPath(); c.arc(X(idx), Y(y[idx]), 3, 0, Math.PI * 2); c.fill();
302 }
303
304 c.fillStyle = "#9ca3af";
305 c.font = "11px -apple-system, system-ui, sans-serif";
306 c.fillText("0 s", 2, h - 3);
307 const secs = (n / (r.fs || 30)).toFixed(0);
308 c.fillText(`${secs} s`, w - 26, h - 3);
309 }
310
311 /* ----- camera */
312
313 async function startCamera() {
314   if (S.recording) return;
315   if (!navigator.mediaDevices?.getUserMedia) {
316     alert("This browser will not give a page camera access.");
317     return;
318   }
319   const main = $("main");
320   main.innerHTML = "";
321   const card = el("div", "card");
322   const head = el("div", "card-head");
323   const h = el("div");
324   h.appendChild(el("h2", null, "Recording"));
325   h.appendChild(el("p", null, "Cover the rear camera and its flash completely with a fingertip. Rest your hand on something solid
and hold still – movement is what the tool refuses most often.));
326   head.appendChild(h);
327   card.appendChild(head);
328
329   const cam = el("div", "cam");
330   const video = el("video");
331   video.autoplay = true; video.playsInline = true; video.muted = true;
332   cam.appendChild(video);
333   const side = el("div");
334   const note = el("div", "cam-note");
335   note.appendChild(el("strong", null, "Waiting for the camera..."));
336   side.appendChild(note);
337   const prog = el("div", "progress");
338   const bar = el("i");
339   prog.appendChild(bar);
340   side.appendChild(prog);
341   cam.appendChild(side);
342   card.appendChild(cam);
343   main.appendChild(card);
344
345   let stream;
346   try {
347     stream = await navigator.mediaDevices.getUserMedia({
348       video: { facingMode: { ideal: "environment" }, width: { ideal: 640 }, height: { ideal: 480 } },
349       audio: false,
350     });
351   } catch (err) {
352     note.textContent = `The camera was not available: ${err.message}`;
353     return;
354   }
355   video.srcObject = stream;
356   const track = stream.getVideoTracks()[0];
357   // The flash is what makes a fingertip translucent enough to read. Not every
358   // device exposes it, and a request that fails is not worth abandoning over.
359   try { await track.applyConstraints({ advanced: [{ torch: true }] }); } catch { /* no torch */ }
360
361   await video.play().catch(() => {});
362   S.recording = true;
363   $("camera").disabled = true;
364
365   const SECONDS = 30;

```

```

366 const cnv = document.createElement("canvas");
367 cnv.width = 64; cnv.height = 64;
368 const ctx = cnv.getContext("2d", { willReadFrequently: true });
369 const values = [], timestamps = [];
370 const t0 = performance.now();
371
372 await new Promise(resolve => {
373   const frame = () => {
374     const now = performance.now();
375     const secs = (now - t0) / 1000;
376     if (secs >= SECONDS) return resolve();
377     if (video.videoWidth) {
378       // A small central crop: the edges of the frame catch stray light that
379       // has not passed through the finger.
380       const s = Math.min(video.videoWidth, video.videoHeight) * 0.4;
381       ctx.drawImage(video, (video.videoWidth - s) / 2, (video.videoHeight - s) / 2, s, s, 0, 0, 64, 64);
382       const d = ctx.getImageData(0, 0, 64, 64).data;
383       let g = 0;
384       for (let i = 1; i < d.length; i += 4) g += d[i];
385       // green tracks haemoglobin absorption better than red, which saturates
386       values.push(g / (d.length / 4));
387       timestamps.push(now - t0);
388     }
389     bar.style.width = `${(secs / SECONDS) * 100}%`;
390     note.innerHTML = "";
391     note.appendChild(el("strong", null, `${(SECONDS - secs).toFixed(0)} seconds left`));
392     note.appendChild(el("div", null, `${values.length} frames captured`));
393     requestAnimationFrame(frame);
394   };
395   requestAnimationFrame(frame);
396 });
397
398 stream.getTracks().forEach(t => t.stop());
399 S.recording = false;
400 $("camera").disabled = false;
401
402 if (values.length < 60) {
403   note.textContent = "Too few frames arrived to measure anything.";
404   return;
405 }
406 measure({
407   values, timestamps, fromCamera: true,
408   label: "Your camera",
409   note: `${values.length} frames over ${((timestamps.at(-1)) / 1000).toFixed(1)} s`,
410 });
411 }
412
413 /* ----- static readings */
414
415 function viewMethod(main) {
416   const card = el("div", "card");
417   const head = el("div", "card-head");
418   const h = el("div");
419   h.appendChild(el("h2", null, "How the measurement works"));
420   h.appendChild(el("p", null, "Arithmetic, not a model. Every step can be checked by hand.));
421   head.appendChild(h);
422   card.appendChild(head);
423   const p = el("div", "prose");
424   p.innerHTML = `
425     <p>A fingertip over a lit camera changes brightness slightly with every heartbeat, as
426     blood fills and leaves the tissue. The green channel carries that change most clearly.
427     Reading a rate from it is easy. Knowing when the reading is wrong is the hard part,
428     and that is what most of this pipeline does.</p>
429
430     <h3>The pipeline</h3>
431     <ul>
432       <li><strong>Resample to 30 Hz.</strong> Camera frames do not arrive evenly. Everything
433       downstream assumes a fixed rate, so the samples are interpolated onto a uniform grid
434       first – which also makes the browser and the Python reproducible against each other.</li>
435       <li><strong>Detrend and bandpass</strong> to ${HR_MIN_BPM}–${HR_MAX_BPM} bpm, a third-order
436       Butterworth applied forwards and backwards so it adds no phase shift.</li>
437       <li><strong>Find beats</strong> as peaks separated by at least a plausible beat interval.</li>
438       <li><strong>Choose a window.</strong> ${WINDOW_SECONDS} seconds slide across the recording and
439       the calmest one is measured. The window is picked by signal purity, never by the rate it
440       produces, so the choice cannot quietly select for a particular answer.</li>

```

```

441     <li><strong>Judge it</strong> against the four checks on the measurement page.</li>
442 </ul>
443
444 <h3>The check that matters most</h3>
445 <div class="callout">Counting beats and reading the frequency spectrum are two independent
446 ways to get the same number. When a finger moves, the beat detector tends to miss alternate
447 beats and report half the true rate – which looks exactly like a genuinely slow pulse and
448 cannot be spotted from the rate alone. Requiring the two methods to agree within
449  $\{(AGREEMENT\_TOL * 100).toFixed(0)\}\%$  is what separates them. On the validation corpus this
450 check alone refuses 117 recordings, and those are disproportionately the ones the tool
451 would otherwise have got badly wrong.</div>
452
453 <h3>Why the same code runs in both places</h3>
454 <p>The numbers on the evidence page were produced by <code>ppg.py</code>. The numbers in this
455 page come from <code>dsp.js</code>. If those two drifted apart, the published accuracy would
456 describe software nobody is running. <code>verify_dsp.mjs</code> runs both over real recordings
457 and compares the filtered signal, the beat positions, the rate, the purity and the accept
458 decision. Each demo recording also carries the values Python produced, and the measurement
459 page shows them next to what your browser just computed.</p>`;
460 groupSections(p);
461 card.appendChild(p);
462 main.appendChild(card);
463 }
464
465 function viewEvidence(main) {
466   const v = S.validation;
467   const card = el("div", "card");
468   const head = el("div", "card-head");
469   const h = el("div");
470   h.appendChild(el("h2", null, "What was measured, and against what"));
471   h.appendChild(el("p", null, v
472     ? `${v.records_analysed.toLocaleString()} smartphone recordings from ${v.dataset}, each with a simultaneous ECG. The reference
rate comes from ${v.reference}.`
473     : "Validation numbers are unavailable."));
474   head.appendChild(h);
475   card.appendChild(head);
476
477   if (v) {
478     const stats = el("div", "stats");
479     const a = v.heart_rate_on_accepted, g = v.quality_gate;
480     stats.appendChild(stat("Recordings", v.records_analysed.toLocaleString(), "", false));
481     stats.appendChild(stat("Refused correctly", (g.specificity * 100).toFixed(1), "%", false));
482     stats.appendChild(stat("Within 5 bpm", (a.within_5_bpm * 100).toFixed(1), "% of accepted", false));
483     stats.appendChild(stat("Median error", a.median_bpm.toFixed(2), "bpm", false));
484     card.appendChild(stats);
485
486     const p = el("div", "prose");
487     p.innerHTML = `
488       <h3>The quality gate</h3>
489       <p>Of ${v.records_analysed.toLocaleString()} recordings, expert annotators called
490       ${g.annotator_good.toLocaleString()} usable. The tool accepts ${a.n}. It refuses
491        $\{(g.specificity * 100).toFixed(1)\}\%$  of what the annotators refuse, and accepts only
492        $\{(g.sensitivity * 100).toFixed(1)\}\%$  of what they accept – deliberately far stricter than
493       a human grader, because a refusal costs a retake and a wrong rate costs trust.</p>
494
495       <h3>Rate accuracy on what it accepts</h3>
496       <p>Mean absolute error ${a.mae_bpm} bpm, median ${a.median_bpm} bpm,
497        $\{(a.within_5_bpm * 100).toFixed(1)\}\%$  within 5 bpm and
498        $\{(a.within_10_bpm * 100).toFixed(1)\}\%$  within 10 bpm of the ECG. The mean sits well above
499       the median because a handful of accepted recordings are badly wrong rather than slightly
500       wrong; that tail is the honest weakness here, and it is reported rather than trimmed.</p>
501
502       <h3>Accuracy is limited by length, not by method</h3>
503       <p>Holding the gate fixed and varying only how much signal it sees, the share of rates
504       landing within 5 bpm of the ECG runs:</p>
505       <ul>${Object.entries(v.duration_trend_within_5bpm)
506         .map(([k, val]) => `<li><code>${k}</code> window –  $\{(val * 100).toFixed(0)\}\%$  within 5 bpm</li>`).join("")}</ul>
507       <p>Every recording in this corpus is ten seconds long, which is the ceiling of that
508       table. The app records thirty. That the trend continues is an extrapolation, not a
509       measurement, and it is listed on the limits page as such.</p>
510
511       <h3>Measuring the calmest window helps</h3>
512       <p>Sliding an eight-second window and measuring the calmest stretch, rather than the first
513       one, raised the number of usable recordings from
514       ${v.window_selection.fixed_8s.n} to ${v.window_selection.cleanest_8s.n} while

```

```

515     <em>also</em> improving mean error from ${v.window_selection.fixed_8s.mae} to
516     ${v.window_selection.cleanest_8s.mae} bpm.</p>
517
518     <h3>On the reference</h3>
519     <p>The dataset ships its own heart-rate column, and this project does not use it. That
520     column is doubled for one subject and differs from the ECG's own R-peak annotations by
521     2.98 bpm on average. Scoring against it charges the tool for the reference's mistakes as
522     well as its own, so the R-peaks are used instead.</p>`;
523     groupSections(p);
524     card.appendChild(p);
525   }
526   main.appendChild(card);
527 }
528
529 function viewLimits(main) {
530   const card = el("div", "card");
531   const head = el("div", "card-head");
532   const h = el("div");
533   h.appendChild(el("h2", null, "What this is not"));
534   h.appendChild(el("p", null, "Finished work and planned work, kept apart."));
535   head.appendChild(h);
536   card.appendChild(head);
537   const p = el("div", "prose");
538   const limits = (S.validation?.honest_limits || []).map(l => `<li>${l}</li>`).join("");
539   p.innerHTML = `
540   <div class="callout"><strong>This is not a medical device and does not diagnose anything.</strong>
541   It reports a pulse rate and a description of how evenly spaced the beats were, or it reports
542   nothing at all.</div>
543
544   <h3>Measured limits</h3>
545   <ul>${limits}</ul>
546
547   <h3>Finished</h3>
548   <ul>
549     <li>The signal pipeline, validated on ${S.validation?.records_analysed ?? 0}.toLocaleString()} recordings against
simultaneous ECG.</li>
550     <li>The quality gate and its four checks.</li>
551     <li>Camera capture in the browser, and the same arithmetic running in Python and in JavaScript, verified against each other.
</li>
552   </ul>
553
554   <h3>Not finished</h3>
555   <ul>
556     <li>Any validation of the irregularity measures as a screen for atrial fibrillation. The
557     corpus has no rhythm labels, so <code>mssd</code> and the interval spread are computed and
558     described but never interpreted.</li>
559     <li>Any measurement on recordings longer than ten seconds. The thirty-second capture is
560     built and works; its accuracy is inferred from the duration trend, not established.</li>
561     <li>Any testing across skin tones, nail polish, cold hands, or camera hardware. The corpus
562     is one cohort recorded on a small number of devices, and PPG is known to behave differently
563     across skin tones. Nothing here characterises that.</li>
564   </ul>`;
565   groupSections(p);
566   card.appendChild(p);
567   main.appendChild(card);
568 }
569
570 const cap = s => s ? s[0].toUpperCase() + s.slice(1) : s;
571
572 /* Two-column prose balances by element, which happily strands a heading at the
573    foot of one column and its list at the head of the next. Grouping everything
574    under a heading into one block keeps each section whole. */
575 function groupSections(root) {
576   const kids = [...root.childNodes];
577   let section = null;
578   for (const node of kids) {
579     if (node.nodeName === "H3") {
580       section = document.createElement("section");
581       root.insertBefore(section, node);
582     }
583     if (section) section.appendChild(node);
584   }
585 }
586
587 boot();

```

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>Pulse – a pulse reading that knows when it is wrong</title>
7 <link rel="stylesheet" href="app.css">
8 </head>
9 <body>
10
11 <header class="bar">
12   <div class="brand"><span class="mark">P</span><span class="word">Pulse</span></div>
13   <nav class="tabs">
14     <button data-view="measure" class="on">Measure</button>
15     <button data-view="method">Method</button>
16     <button data-view="evidence">Evidence</button>
17     <button data-view="limits">Limits</button>
18   </nav>
19   <div class="bar-right">
20     <span class="pill-ghost" id="srcpill">Recorded trace</span>
21     <span class="chip">PPG · 30 Hz</span>
22   </div>
23 </header>
24
25 <div class="toolbar">
26   <div class="toolbar-left">
27     <label class="select">
28       <span class="select-icon">▣</span>
29       <select id="sample"></select>
30     </label>
31     <span class="tool-note" id="toolnote">Four recordings from BUT PPG, each with a simultaneous ECG</span>
32   </div>
33   <div class="toolbar-right">
34     <button class="btn-ghost" id="replay" title="Measure again">↺</button>
35     <button class="btn-primary" id="camera"><span class="spark">●</span> Use my camera</button>
36   </div>
37 </div>
38
39 <main id="main"></main>
40
41 <footer class="status">
42   <span class="legend"><i class="d-ok"></i> accepted</span>
43   <span class="legend"><i class="d-no"></i> refused</span>
44   <span class="legend"><i class="d-beat"></i> detected beat</span>
45   <span class="legend"><i class="d-win"></i> window measured</span>
46   <span class="status-right" id="statusright">nothing measured yet</span>
47 </footer>
48
49 <script type="module" src="app.js"></script>
50 </body>
51 </html>
```

```
1 :root {
2   --indigo: #4338ca;
3   --indigo-deep: #3730a3;
4   --indigo-soft: #ede9fe;
5   --lav: #ddd9f8;
6   --page: #f4f5f7;
7   --card: #ffffff;
8   --line: #e5e7eb;
9   --ink: #111827;
10  --grey: #6b7280;
11  --grey-2: #9ca3af;
12  --ok: #15803d;
13  --ok-bg: #e7f6ec;
14  --warn: #b45309;
15  --warn-bg: #fef3e2;
16  --no: #b91c1c;
17  --no-bg: #fdecec;
18 }
19
20 * { box-sizing: border-box; }
21
22 html, body {
23   margin: 0;
24   background: var(--page);
25   color: var(--ink);
26   font: 14px/1.5 -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
27 }
28
29 /* ----- top bar */
30
31 .bar {
32   display: flex;
33   align-items: center;
34   gap: 22px;
35   padding: 0 18px;
36   height: 54px;
37   background: linear-gradient(90deg, var(--indigo-deep), var(--indigo));
38   color: #fff;
39 }
40
41 .brand { display: flex; align-items: center; gap: 9px; }
42
43 .mark {
44   display: grid;
45   place-items: center;
46   width: 26px; height: 26px;
47   border-radius: 7px;
48   background: #fff;
49   color: var(--indigo);
50   font-weight: 700;
51   font-size: 14px;
52 }
53
54 .word { font-weight: 650; font-size: 16px; letter-spacing: -0.01em; }
55
56 .tabs { display: flex; gap: 4px; }
57
58 .tabs button {
59   border: 0;
60   background: transparent;
61   color: rgba(255, 255, 255, 0.78);
62   font: inherit;
63   font-weight: 500;
64   padding: 7px 14px;
65   border-radius: 999px;
66   cursor: pointer;
67 }
68
69 .tabs button:hover { color: #fff; background: rgba(255, 255, 255, 0.12); }
70 .tabs button.on { background: rgba(255, 255, 255, 0.18); color: #fff; font-weight: 600; }
71
72 .bar-right { margin-left: auto; display: flex; align-items: center; gap: 10px; }
```

```

73
74 .pill-ghost {
75   padding: 5px 13px;
76   border-radius: 999px;
77   border: 1px solid rgba(255, 255, 255, 0.35);
78   font-size: 12.5px;
79   font-weight: 500;
80 }
81
82 .chip {
83   padding: 5px 12px;
84   border-radius: 999px;
85   background: rgba(255, 255, 255, 0.14);
86   font-size: 12.5px;
87   font-variant-numeric: tabular-nums;
88 }
89
90 /* ----- toolbar */
91
92 .toolbar {
93   display: flex;
94   align-items: center;
95   gap: 12px;
96   padding: 11px 18px;
97   background: #fff;
98   border-bottom: 1px solid var(--line);
99 }
100
101 .toolbar-left { display: flex; align-items: center; gap: 12px; min-width: 0; }
102 .toolbar-right { margin-left: auto; display: flex; align-items: center; gap: 9px; }
103
104 .select {
105   display: inline-flex;
106   align-items: center;
107   gap: 7px;
108   border: 1px solid var(--line);
109   border-radius: 8px;
110   padding: 0 10px;
111   height: 34px;
112   background: #fff;
113 }
114
115 .select-icon { color: var(--grey-2); font-size: 12px; }
116
117 .select select {
118   border: 0;
119   outline: 0;
120   background: transparent;
121   font: inherit;
122   color: var(--ink);
123   height: 32px;
124   padding-right: 4px;
125   cursor: pointer;
126   max-width: 340px;
127 }
128
129 .tool-note { color: var(--grey); font-size: 12.5px; }
130
131 .btn-primary {
132   display: inline-flex;
133   align-items: center;
134   gap: 7px;
135   height: 34px;
136   padding: 0 15px;
137   border: 0;
138   border-radius: 8px;
139   background: var(--indigo);
140   color: #fff;
141   font: inherit;
142   font-weight: 600;
143   cursor: pointer;
144 }
145
146 .btn-primary:hover { background: var(--indigo-deep); }
147 .btn-primary.disabled { background: var(--grey-2); cursor: default; }

```

```

148 .spark { font-size: 11px; }
149
150 .btn-ghost {
151   width: 34px; height: 34px;
152   border: 1px solid var(--line);
153   border-radius: 8px;
154   background: #fff;
155   color: var(--grey);
156   font-size: 15px;
157   cursor: pointer;
158 }
159
160 .btn-ghost:hover { color: var(--ink); border-color: var(--grey-2); }
161
162 /* ----- main */
163
164 main { padding: 16px 18px 64px; display: grid; gap: 16px; }
165
166 .card {
167   background: var(--card);
168   border: 1px solid var(--line);
169   border-radius: 12px;
170   overflow: hidden;
171 }
172
173 .card-head {
174   display: flex;
175   align-items: flex-start;
176   gap: 16px;
177   padding: 16px 18px;
178 }
179
180 .card-head h2 { margin: 0 0 3px; font-size: 15.5px; font-weight: 650; }
181 .card-head p { margin: 0; color: var(--grey); font-size: 13px; max-width: 70ch; }
182 .head-right { margin-left: auto; flex: none; }
183
184 .verdict {
185   display: inline-flex;
186   align-items: center;
187   gap: 7px;
188   padding: 6px 13px;
189   border-radius: 999px;
190   font-size: 12.5px;
191   font-weight: 600;
192 }
193
194 .verdict.ok { background: var(--ok-bg); color: var(--ok); }
195 .verdict.no { background: var(--warn-bg); color: var(--warn); }
196
197 /* the divided stat strip */
198
199 .stats {
200   display: grid;
201   grid-template-columns: repeat(4, 1fr);
202   border-top: 1px solid var(--line);
203   border-bottom: 1px solid var(--line);
204 }
205
206 .stat { padding: 14px 18px; border-right: 1px solid var(--line); }
207 .stat:last-child { border-right: 0; }
208
209 .stat-k {
210   font-size: 10.5px;
211   letter-spacing: 0.08em;
212   text-transform: uppercase;
213   color: var(--grey);
214   font-weight: 600;
215 }
216
217 .stat-v {
218   margin-top: 5px;
219   font-size: 26px;
220   font-weight: 600;
221   font-variant-numeric: tabular-nums;
222   letter-spacing: -0.02em;

```

```

223 }
224
225 .stat-v.dim { color: var(--grey-2); }
226 .stat-u { font-size: 13px; color: var(--grey); font-weight: 500; margin-left: 3px; }
227
228 /* chart */
229
230 .plot { padding: 14px 18px 10px; }
231 .plot canvas { width: 100%; height: 210px; display: block; }
232
233 .plot-legend {
234   display: flex;
235   gap: 18px;
236   padding: 2px 18px 14px;
237   color: var(--grey);
238   font-size: 12px;
239 }
240
241 .plot-legend span { display: inline-flex; align-items: center; gap: 7px; }
242 .sw { width: 13px; height: 3px; border-radius: 2px; background: var(--indigo); }
243 .sw.fill { height: 11px; background: var(--lav); border-radius: 3px; }
244 .sw.dot { width: 7px; height: 7px; border-radius: 50%; background: var(--indigo); }
245
246 /* the checks table */
247
248 .rows { border-top: 1px solid var(--line); }
249
250 .row-head {
251   display: grid;
252   grid-template-columns: 210px 1fr 108px;
253   gap: 14px;
254   padding: 9px 18px;
255   background: #fafafa;
256   border-bottom: 1px solid var(--line);
257   color: var(--grey);
258   font-size: 11px;
259   letter-spacing: 0.06em;
260   text-transform: uppercase;
261   font-weight: 600;
262 }
263
264 .row {
265   display: grid;
266   grid-template-columns: 210px 1fr 108px;
267   gap: 14px;
268   padding: 12px 18px;
269   border-bottom: 1px solid var(--line);
270   align-items: center;
271 }
272
273 .row:last-child { border-bottom: 0; }
274 .row-name { font-weight: 550; }
275 .row-why { color: var(--grey); font-size: 13px; }
276
277 .tag {
278   display: inline-block;
279   padding: 3px 10px;
280   border-radius: 6px;
281   font-size: 12px;
282   font-weight: 600;
283   font-variant-numeric: tabular-nums;
284 }
285
286 .tag.pass { background: var(--ok-bg); color: var(--ok); }
287 .tag.fail { background: var(--no-bg); color: var(--no); }
288 .tag.skip { background: #f3f4f6; color: var(--grey-2); }
289
290 /* prose blocks for method / limits */
291
292 /* Long-form sections read as one narrow column on a phone and two on a desk,
293    so a wide window does not leave half the card empty. */
294 .prose { padding: 4px 18px 20px; max-width: 78ch; }
295 .prose h3 { font-size: 14px; margin: 20px 0 7px; break-after: avoid; }
296 .prose h3:first-child { margin-top: 6px; }
297 .prose p { margin: 0 0 11px; color: #374151; break-inside: avoid; }

```

```

298 .prose ul { margin: 0 0 11px; padding-left: 19px; color: #374151; break-inside: avoid; }
299
300 @media (min-width: 1100px) {
301   .prose { max-width: none; columns: 2; column-gap: 44px; }
302   .prose section { break-inside: avoid; }
303   .prose > :first-child { margin-top: 0; }
304 }
305 .prose li { margin-bottom: 5px; }
306 .prose code {
307   background: #f3f4f6;
308   padding: 1px 5px;
309   border-radius: 4px;
310   font-size: 12.5px;
311   font-family: ui-monospace, SFMono-Regular, Menlo, monospace;
312 }
313
314 .callout {
315   break-inside: avoid;
316   border-left: 3px solid var(--indigo);
317   background: #faf9ff;
318   padding: 11px 14px;
319   border-radius: 0 8px 8px 0;
320   margin: 0 0 13px;
321   color: #374151;
322 }
323
324 /* camera panel */
325
326 .cam { display: grid; grid-template-columns: 220px 1fr; gap: 18px; padding: 16px 18px; }
327 .cam video { width: 100%; border-radius: 10px; background: #111827; aspect-ratio: 3/4; object-fit: cover; }
328 .cam-note { color: var(--grey); font-size: 13px; }
329 .cam-note strong { color: var(--ink); }
330
331 .progress { height: 6px; background: var(--line); border-radius: 999px; overflow: hidden; margin: 12px 0 8px; }
332 .progress i { display: block; height: 100%; background: var(--indigo); width: 0; }
333
334 /* ----- status bar */
335
336 .status {
337   position: fixed;
338   left: 0; right: 0; bottom: 0;
339   display: flex;
340   align-items: center;
341   gap: 20px;
342   padding: 0 18px;
343   height: 36px;
344   background: #1f2330;
345   color: #cbd0dc;
346   font-size: 12px;
347 }
348
349 .legend { display: inline-flex; align-items: center; gap: 7px; }
350 .legend i { width: 8px; height: 8px; border-radius: 50%; }
351 .d-ok { background: #4ade80; }
352 .d-no { background: #f87171; }
353 .d-beat { background: #a5b4fc; }
354 .d-win { background: #ddd9f8; }
355 .status-right { margin-left: auto; color: #8b93a7; font-variant-numeric: tabular-nums; }
356
357 @media (max-width: 860px) {
358   .stats { grid-template-columns: repeat(2, 1fr); }
359   .stat:nth-child(2) { border-right: 0; }
360   .cam { grid-template-columns: 1fr; }
361   .row, .row-head { grid-template-columns: 1fr 84px; }
362   .row-why { display: none; }
363   .tabs button { padding: 7px 10px; }
364   .tool-note { display: none; }
365 }

```